Конфіденційно

Vision — аудит безпеки

Третя ітерація: після специфікацій 051 і 052, перед перенесенням у банківський контур

Коміт	`fe7a82c` (`main` після PR #34); попередня ітерація — `a3b6104` (2026-11-07), перша — `7b13af1` (2026-11-05)
Дата	8 листопада 2026
Скоуп	монорепо `nazarenkod/vision`: backend, prisma, frontend, CLI/MCP, sdk-java, scripts, infra, CI
Модель розгортання	внутрішній on-prem сервіс; кілька команд в одному інстансі; на тій самій машині, що й робочий VRT
Промт	`audit/REVIEW-PROMPT.md` (без змін); попередні звіти — `report-2026-11-07.pdf`, `report-2026-11-05.pdf`
Що не оцінювалось	рішення власника й відкладені задачі (розділ 4.2) — за вказівкою власника
Класифікація	Конфіденційно — лише для керівництва, ІБ та розробників

Зміст

1. Резюме для керівництва і вердикт
2. Скоуп, методологія та обмеження
3. Архітектура та модель загроз
4. Регресія попередніх знахідок і що виключено
5. Зведена таблиця відкритих знахідок
6. Детальні знахідки
7. Дані за межами контуру та LLM
8. Комерційне використання, ліцензії й банківський контур
9. Roadmap
10. Додатки

1. Резюме для керівництва і вердикт

Це третя перевірка Vision. Після попередньої ітерації (2026-11-07) змерджено дві специфікації безпеки: **051** (секрети, Jira, ліміт областей ігнорування, лінійний рушій виразів) і **052** (сесії на сервері, SSO лише у своїй вкладці, CSP і HSTS). Аудит виконано заново за тим самим промтом. **Рішення власника й відкладені на майбутнє задачі не оцінювались** (перелік — розділ 4.2).

Вердикт: ще не готовий до перенесення — два блокери, обидва невеликі за обсягом виправлення.

Обидва блокери — того самого класу, що й VIS-SEC-043 минулого разу, яку тоді вважали блокером: будь-який учасник проєкту (навіть роль `guest` чи API-ключ CI) одним запитом зупиняє API для **всіх команд** на секунди-хвилини, а серією запитів — валить процес через нестачу пам'яті. Обидва відтворено на стенді.

1. **VIS-SEC-058 (High).** Новий «лінійний» рушій виразів зі специфікації 051 (RE2JS) справді не має експоненти, але приймає вирази на 300 символів, які компілюються в сотні тисяч інструкцій: один вираз — +46 МіБ у кеші (кеш на 500 виразів) і 1–7 с головного потоку. Через `GET /test-runs/:id/selector-matches` це доступно guest-у. Мета US-4 специфікації 051 (повільний вираз однієї команди не зупиняє API) не досягнута.
2. **VIS-SEC-043 (High, відкрито знову).** Виправлення 2026-11-07 закрило лише один квадратичний шлях. Список, де рядки мають однаковий `resource-id` (типово для RecyclerView), знову квадратичний: 8000 рядків (2,2 МБ XML) — 18,8 с головного потоку на відкриття екрана.

Окремо **специфікація 052 має дефект реалізації** (VIS-SEC-059, Medium): збереження профілю видає новий токен, а старий не завершує, тож вихід після цього не гасить раніше скопійований токен — саме той сценарій, який 052 мала закрити. І **зміна пароля не питає поточного пароля** (VIS-SEC-060, Medium): після 052 той, хто вкрав токен, змінює пароль, лишається увійденим, а власника виганяє з усіх браузерів. Обидва відтворено на стенді.

High 2 **Medium 10** **Low 30** **Info 8** **Закрито 13**

Відкритих знахідок — **50** (Critical 0). Із них 12 нових: High 1, Medium 3, Low 5, Info 3; одна стара (043) відкрита знову. Ще 6 знахідок не оцінювались як рішення власника.

Що змінилось від 2026-11-07.

- **Закрито в цій ітерації й підтверджено:** VIS-SEC-002 (слабкі секрети), 005 (області ігнорування з запиту), 008 (allowlist Jira), 009 (сесії — крім дефекту 059), 015 (ReDoS на `RegExp`), 018 (login CSRF через SSO), 047 (токен прив'язаний до email), 048 (Jira types). Разом із закритими раніше (001, 007, 012, 014, 045) — 13.
- **Регресії виправлень:** 058 (рушій, яким закрито 015), 043 (повторно), 059 (сесії 052), 061 (ліміт 20 областей обходиться областями з дерева).
- **Без змін:** решта Medium з попередньої ітерації — 004, 010, 016, 017, 019, 021, 046 — не входили в 051/052 і не мають рішення власника.
- `npm audit` = 0 у трьох пакетах (prod і dev). Секретів у дереві немає. Ліцензії в поставці — лише пермісивні.
- Unit бекенду 929/929, e2e бекенду 91/91 на стенді, unit фронтенду 274/274, `check-docs` — ок.

Топ-5 до перенесення.

#	Знахідка	Severity	Зусилля
1	VIS-SEC-058 — обмежити розмір програми виразу, DTO з `MaxLength` для `selector-matches`, кеш за байтами	High	M
2	VIS-SEC-043 — не перевіряти варіант повним XPath; `/structure` у воркер з тайм-аутом	High	S–M
3	VIS-SEC-059 — `PUT /users` не відкриває новий вхід (або завершує поточний)	Medium	S
4	VIS-SEC-060 — поточний пароль для зміни пароля (і email у режимі БД)	Medium	S–M
5	VIS-SEC-061 — загальна межа областей після злиття (запит + дерево + правила)	Medium	S

Після цих п'яти (оцінка — 2–3 дні з тестами) рекомендований вердикт — **готовий за умови** виконання пунктів «до перенесення» з розділу 9 (дефолтний пароль адміна 010 і вимикач AI 016 — конфігурацією поставки).

2. Скоуп, методологія та обмеження

Скоуп: увесь код на коміті `fe7a82c`, з окремою увагою до диффу `a3b6104..fe7a82c` (143 файли, специфікації 051 і 052). Робочий VRT, його база, контейнери й картинки — поза скоупом і не чіпались. Реальні Jira, Gemini, Google Drive — не викликались.

Методологія.

- **Етап 1 — сканування** (сирі виводи — `audit/raw/*-2026-11-08*`): `npm audit` prod і dev у трьох пакетах; semgrep 1.x з 977 локальних правил спільноти (javascript, typescript, java, python, dockerfile, bash, secrets, github-actions; без реєстру semgrep.dev); bandit по CLI; shellcheck по `scripts/`; власний regex-сканер секретів по всіх відстежуваних файлах (gitleaks запустити не дозволив запобіжник середовища); підрахунок ліцензій з `package.json` кожного встановленого пакета.
- **Етап 2 — ручний аудит** чотирма паралельними агентами за розділами промту: A — автентифікація й сесії (2.1), B — авторизація, ізоляція, ін'єкції (2.2, 2.5; таблиця 88 маршрутів — `raw/endpoints-2026-11-08.md`), C — знімки, дерева, DoS, LLM, SSRF (2.3, 2.4), D — HTTP-шар, інфраструктура, supply chain, ліцензії (2.6–2.8). Кожна знахідка High/Medium перевірена мною окремо; три ключові — наживо.
- **Стенд:** нативний Postgres 16 на `127.0.0.1:55434` із синтетичною базою, Node 24.11, випадкові `JWT_SECRET` і `VISION_SECRET_KEY`. Відтворення — тимчасовими e2e-файлами, видаленими після прогону: `raw/repro-008-sessions-profile-password.txt`, `raw/repro-009-re2-cost.txt`, `raw/repro-010-structure-same-id.txt`.

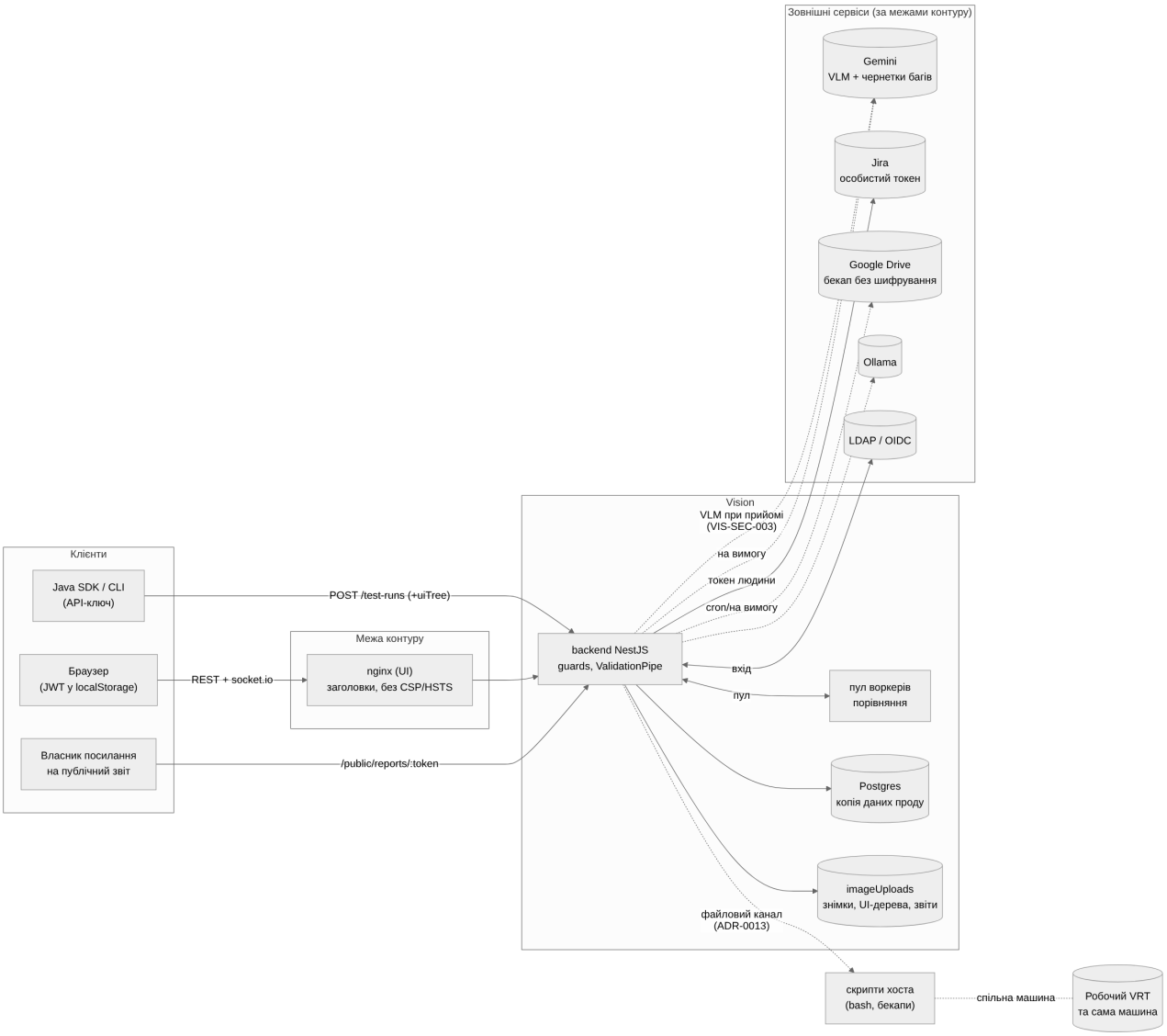
Обмеження.

- **Без Docker-демона** — немає `trivy image`, compose і nginx-образ не запускались; заголовки nginx перевірено за конфігом і `env.sh`.
- **trivy, osv-scanner, gitleaks, hadolint, syft** — бінарники з GitHub Releases завантажено, але запобіжник середовища не дозволив їх виконати. Замінено: `npm audit` (вразливості npm), власний regex-сканер (секрети), ручний огляд Dockerfile (агент D). SBOM не перегенеровано syft-ом: `audit/sbom.cdx.json` — з 2026-11-07 (805 компонентів), з тих пір додано одну залежність — `re2js@2.9.1` (MIT, без залежностей), її дописано вручну (806). Gradle — без транзитивного графа (мережа), як і раніше.
- Без історії git (рішення власника). LDAP/IdP не піднімались — LDAP/OIDC-знахідки з коду.
- Playwright не запускався (браузерні тести не змінюють висновків безпеки).
- Вимірювання DoS — короткі (до ~20 с), далі — екстраполяція з кількох точок.

3. Архітектура та модель загроз

Модель загроз і діаграма потоків — `audit/threat-model.md` і `audit/assets/dfd.png` (без змін складу компонентів). Після 052 додано один актив і одну межу:

- **Сесії** (`Session`, `sid` у JWT) — тепер відкликання входу залежить від того, що кожен токен має живий рядок сесії. Загрози: сесії, які ніхто не завершує (VIS-SEC-059), необмежене зростання таблиці (VIS-SEC-062).
- **Вкладка браузера як межа SSO** (`nonce` у `sessionStorage`) — закрила login CSRF (VIS-SEC-018).



Головна загроза цієї ітерації — **доступність спільного інстансу**: обчислення над даними, які задає клієнт (вирази правил, XPath, дерева), виконуються в головному потоці API, спільному для всіх команд і для сокета подій. Три з чотирьох нових Medium/High — саме тут.

4. Регресія попередніх знахідок і що виключено

4.1. Закрито й підтверджено на HEAD

ID	Знахідка	Було	Підтвердження
VIS-SEC-001	Масове присвоєння в PATCH /builds/:id: ескалація editor→admin і крадіжка даних між командами	Critical	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-002	Приймаються відомі/слабкі секрети підпису; DEPLOY.md радить секрет робочого VRT	High	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-005	Області ігнорування без меж координат зависають воркер порівняння без тайм-ауту	Medium	backend/src/compare/utils/index.ts:14-28 (Math.max(0, area.x/y)); dto/ignore-area.dto.ts:6 MAX_IGNORE_AREAS=20 + @ArrayMaxSize на create, multipart, update, add (test-runs.service.…
VIS-SEC-007	Вхід через LDAP з порожнім паролем не відхиляється (ризик unauthenticated bind)	High	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-008	Allowlist Jira перевіряє лише hostname: проходять http, будь-який порт і редиректи	Medium	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-009	Немає відкликання сесій: logout, зміна пароля, блокування й зміна ролі не гасять JWT і сокет	Medium	Сесії 052 працюють: вихід, зміна пароля, пароль від адміна, видалення, зміна ролі — і в API, і в сокетах. Новий баг реалізації винесено окремо як VIS-SEC-059 (PUT /users відкриває …
VIS-SEC-012	Глобальний ValidationPipe без whitelist/forbidNonWhitelisted/transform (системний корінь mass assignment)	Medium	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)

ID	Знахідка	Було	Підтвердження
VIS-SEC-014	Немає меж розміру файла й розмірів PNG: multer без limits, повне декодування в головному потоці	Medium	backend/src/test-runs/test-runs.controller.ts:384 FileInterceptor('image', {limits:{fileSize: maxBytes, files:1}}); test-runs.service.ts:93-98 screenshotProblem до будь-якого запису…
VIS-SEC-015	ReDoS: регулярний вираз із text-селектора правила елемента виконується над текстами дерева в головному потоці	Medium	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-018	Login CSRF / фіксація сесії: /login#token=<будь-який JWT> входить без state	Medium	frontend/src/pages/LoginPage.tsx:38-42 takesOidcToken; frontend/src/services/users.service.ts newOidcNonce/startOidcLogin/takesOidcToken (sessionStorage, одноразово); backend users…
VIS-SEC-045	Обхід ліміту пікселів: перевіряється лише перший IHDR, а pngjs розпаковує інше (повторний IHDR, interlace-бомба, канва різних розмірів)	Medium	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)
VIS-SEC-047	JWT прив'язаний лише до змінного email: у режимі БД можна заздалегідь «накарбувати» токен для чужої майбутньої адреси	Low	backend/src/auth/sessions.ts:37-40 — токен із sid приймається лише якщо session.userId === user.id; auth.service.ts: 40-49 кожен новий токен має sid — «Накарбований» на чужу майбутн…
VIS-SEC-048	Новий GET /projects/:id/jira/types надсилає особистий токен Jira на адресу з query-рядка; перевірка проєкту декоративна (розширення VIS-SEC-008)	Low	закрито в попередній ітерації; на HEAD не змінювалось і не відкрито знову (тести 051/052 проходять)

S-1…S-15 (аудит 2026-11-04): S-1 — дерево без сутностей і з лімітом 10 МиБ (глибина ≤ 100 через `maxNestedTags` fast-xml-parser); S-2/S-3 — секрети зашифровані, API-ключі хешем; S-4 — без `.env` у git, слабкі секрети не стартують (051); S-5 — знімки за підписом; S-6/S-7/S-15 — ліміт входу, пароль ≥ 12, заголовки, CORS, Swagger за прапорцем, помилки без stack; S-8…

S-10 — образи без root і за digest; S-11 — ізоляція проєктів. Обходів не знайдено. S-12, S-13, S-14 — рішення власника, не оцінювались.

4.2. Не оцінювалось: рішення власника й відкладені задачі

За вказівкою власника ці пункти не рахуються як знахідки цього аудиту. Перевірялось лише, що реалізація самого рішення не має нового бага.

ID	Що	Підстава
VIS-SEC-003	VLM-рушій автоматично надсилає знімки банку в Gemini при прийомі (напруга з правилом 6 / ADR-0007)	прийнятий ризик (рішення власника 2026-11-07): з Gemini є контракт
VIS-SEC-044	Локальний пароль діє для людей каталогу й SSO: паролі з робочого VRT обходять SSO, а звільнені з каталогу входять і самі ставлять собі пароль	відкладено до LDAP-авторизації (рішення власника, spec 052 «Поза межами»)
VIS-SEC-011	Немає CSP і HSTS; поставка публікує HTTP на всіх інтерфейсах без TLS-термінації	залишок (власний https і TLS-термінація в поставці) відкладено — spec 052 «Поза межами»; CSP і HSTS-логіка працюють
VIS-SEC-013	Бекап у Google Drive не шифрується і містить тексти екранів, email і токени публічних звітів	бекапи в Google Drive «лишаються як є» (ADR-0012, bank-readiness-plan)
VIS-SEC-020	SSO повертає роль editor тим, кого адмін понизив до guest; generic 'admin' у типовій групі	залишок (загальна група admin у OIDC_ADMIN_VALUES) — рішення власника 2026-11-08 (REQ-052-13 знято)
VIS-SEC-006	Режим LDAP: будь-який користувач переприв’язує свій акаунт до чужої особи з каталогу	залишок (поле email у профілі в режимі LDAP) — рішення власника 2026-11-07
S-12	OIDC_TLS_NO_VERIFY — лишається опцією	рішення власника / відкладено
S-13	токен у localStorage — окрема зміна (spec 052)	рішення власника / відкладено
S-14	bcrypt cost 10	рішення власника / відкладено
—	ротація секретів без втрати токенів; S3; React 19/MUI 7; SDK з нуля; історія git; блокування акаунта без видалення; перелік сесій	рішення власника / відкладено

5. Зведена таблиця відкритих знахідок

Відкритих — 50: Critical 0, High 2, Medium 10, Low 30, Info 8.

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-043	High	відкрито знову	backend	Квадратична побудова елементів для вибору (structure) блокує event loop на хвилини одним широким UI-деревом
VIS-SEC-058	High	нова	backend	Новий «лінійний» рушій виразів (RE2JS) дуже дорогий: 46–130 МіБ і ~1 с на текст для одного виразу, кеш на 500 виразів; guest може покласти API через selector-matches
VIS-SEC-004	Medium	частково	backend	Обчислювальна DoS: аналіз доступності UI-дерева блокує event loop для всіх команд
VIS-SEC-010	Medium	не виправлено	prisma	Дефолтний адмін із паролем 123456: нижче політики, друкується в лог, без примусової зміни
VIS-SEC-016	Medium	не виправлено	backend	AI_PROVIDER=none не вимикає зовнішній AI: особисті ключі Gemini і ключ проєкту діють далі
VIS-SEC-017	Medium	не виправлено	backend	Адмін задає пароль будь-якому акаунту (зокрема SSO/LDAP) і діє його токеном Jira

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-019	Medium	не виправлено	infra	Dev-compose у дереві: Postgres на 0.0.0.0 з postgres/postgres, ім'я контейнера збігається з продом, моки LDAP/OIDC
VIS-SEC-021	Medium	не виправлено	ci	Запобіжник агентів guard.py легко обходиться — правила 1–4 тримаються лише на дисципліні
VIS-SEC-046	Medium	не виправлено	backend	Розбір XML, серіалізація дерева і декодування PNG при прийомі досі в головному потоці; глибина дерева дає ×200 підсилення
VIS-SEC-059	Medium	нова	backend	Збереження профілю (PUT /users) відкриває новий вхід, а старий токен цього браузера лишається дійсним і після виходу
VIS-SEC-060	Medium	нова	backend	Зміна пароля, email і випуск API-ключа без повторної автентифікації: викрадений токен закріплюється й виганяє власника
VIS-SEC-061	Medium	нова	backend	Області ігнорування, які сервер виводить із дерева (системні панелі, правила), обходять ліміт 20 і займають воркер порівняння до тайм-ауту
VIS-SEC-022	Low	не виправлено	backend	Підписане посилання на знімок безстрокове й не прив'язане до користувача
VIS-SEC-023	Low	не виправлено	backend	Підробка записів у лозі через назву перевірки

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-024	Low	частково	backend	Шифрування секретів без AAD, KDF = один SHA-256; приймається вже зашифроване значення
VIS-SEC-025	Low	не виправлено	backend	Токени публічних звітів і токени Google Drive зберігаються відкрито
VIS-SEC-026	Low	не виправлено	backend	AI-ендпоінти на вимогу доступні ролі guest (платні виклики й дані проєкту)
VIS-SEC-027	Low	не виправлено	backend	У логи потрапляють відповіді моделі (тексти екранів) і URL із підписами/токенами звітів
VIS-SEC-028	Low	не виправлено	cli	CLI пересилає заголовок apiKey при редиректі на інший хост/схему
VIS-SEC-029	Low	не виправлено	frontend	test:pw:install вимикає перевірку TLS під час завантаження браузера
VIS-SEC-030	Low	не виправлено	ci	CI: actions за тегами, не за SHA; не обмежено permissions GITHUB_TOKEN
VIS-SEC-031	Low	не виправлено	sdk-java	Gradle wrapper 6.1 без distributionSha256Sum і без перевіреної контрольної суми jar
VIS-SEC-032	Low	не виправлено	backend	Prompt injection: тексти екрана й назви перевірок у промптах; вердикт VLM сам ставить статус
VIS-SEC-033	Low	не виправлено	backend	Енумерація користувачів за часом відповіді входу

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-034	Low	не виправлено	backend	Ліміт входу лише за email: password spraying і блокування чужих акаунтів
VIS-SEC-035	Low	не виправлено	backend	Звільнені з каталогу/IdP зберігають доступ через API-ключ (безстроковий, без перевірки каталогу)
VIS-SEC-036	Low	частково	backend	PUT /users записує email без нормалізації й перевірки дублікатів в іншому регістрі
VIS-SEC-037	Low	не виправлено	scripts	Скрипти запускають образи за тегом з обліковими даними проду і --network host
VIS-SEC-038	Low	не виправлено	scripts	vision-safety.sh: .env.demo виконується як shell і може перевизначити закріплені VISION_*
VIS-SEC-042	Low	не виправлено	backend	Тестові фікстури містять фотографію реальних людей (успадковано від upstream)
VIS-SEC-049	Low	не виправлено	backend	Лічильник спроб входу: квадратична CPU-DoS і необмежене зростання пам'яті від анонімних запитів
VIS-SEC-050	Low	не виправлено	backend	Фільтр LDAP: String.replaceAll трактує $', $` і $& з email як шаблони заміни — обхід екранування

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-051	Low	не виправлено	backend	LDAP: вимкнення перевірки TLS діє на весь клієнт без попередження, ldap:// без TLS приймається мовчки
VIS-SEC-052	Low	не виправлено	backend	Три падіння воркера поспіль (зокрема повтори одного завдання) назавжди переводять пул порівняння в головний потік без тайм-ауту
VIS-SEC-053	Low	не виправлено	backend	XPath-селектори правил з осями // і ancestor мають необмежену складність і виконуються в головному потоці
VIS-SEC-054	Low	не виправлено	backend	Тексти з UI-дерева і назва перевірки потрапляють у wiki-розмітку Jira без екранування
VIS-SEC-055	Low	не виправлено	backend	VLM-вердикт при прийомі запускає автоапрув і множить виклики Gemini без ліміту
VIS-SEC-062	Low	нова	backend	Таблиця Session росте без меж: рядки чистяться лише при новому вході того самого користувача, а PUT /users додає рядок без bcrypt і ліміту
VIS-SEC-063	Low	нова	backend	Перевірка нестабільного тексту з кількома частинами може мовчки ніколи не спрацювати: кожна частина приймається, а разом вираз не компілюється

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-064	Low	нова	scripts	API (будь-який його компроміс) пише rclone.conf, який хостовий rclone виконує без перевірки — обхід межі ADR-0013
VIS-SEC-065	Low	нова	frontend	frontend/package-lock.json: 162 пакети без resolved/integrity — npm ci не перевіряє їх цілісність
VIS-SEC-066	Low	нова	infra	Postgres демо-стенду з паролем postgres/postgres досяжний з хоста через IP мережі Docker
VIS-SEC-039	Info	не виправлено	infra	Образи не містять LICENSE і NOTICE (Apache-2.0 §4(a), (d))
VIS-SEC-040	Info	не виправлено	backend	JWT без явного алгоритму, issuer і audience
VIS-SEC-041	Info	не виправлено	backend	Editor може змінювати налаштування й видаляти будь-який доступний проєкт (зокрема ключ Gemini)
VIS-SEC-056	Info	не виправлено	backend	StripUnknownFieldsPipe не чистить вкладені DTO: зайві ключі областей ігнорування зберігаються в JSON
VIS-SEC-057	Info	не виправлено	infra	Застарілі облікові дані й чужа IP-адреса в документації та тестах
VIS-SEC-067	Info	нова	infra	Успадкований TLS-конфіг ssl/ без заголовків безпеки й CSP

ID	Severity	Стан	Компонент	Знахідка
VIS-SEC-068	Info	нова	scripts	Звіт аудиту з відкритими вразливостями й репро закомічено і потрапить в експорт у git банку
VIS-SEC-069	Info	нова	frontend	Образ інтерфейсу віддає source maps

6. Детальні знахідки

Нові, відкриті знову й High/Medium — повністю; решта Low/Info без змін від попередньої ітерації — коротко (повний текст — `report-2026-11-07.pdf` і `findings.json`).

VIS-SEC-043 — Квадратична побудова елементів для вибору (structure) блокує event loop на хвилини одним широким UI-деревом

High · **Стан:** відкрито знову · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)` · **CWE:** CWE-407 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M

Де: `backend/src/ui-tree/selector-variants.ts:113-118; backend/src/ui-tree/xpath-selector.ts:186-198; backend/src/ui-tree/ui-tree.service.ts:487`

Перевірка 2026-11-08: Виправлення 2026-11-07 прибрало квадратичний фільтр сиблінгів, але selector-variants.ts:113-118 досі перевіряє кожен варіант повним findBySelector, а anchorsOf (xpath-selector.ts:186-198) повертає всі вузли з тим самим resource-id. Список, де рядки мають один resource-id (звично для RecyclerView), знову квадратичний: 2000 рядків — 0,95 с, 4000 — 4,1 с, 8000 (2,2 МБ XML) — 18,8 с головного потоку (raw/repro-010-structure-same-id.txt). Дерево до 10 МиБ — хвилини, як і до виправлення.

Опис. GET /test-runs/:id/structure (відкриття знімка в інтерфейсі) синхронно в головному потоці викликає pickableElements для дерева і для дерева еталона. Для кожного безіменного видимого вузла selectorVariants будує шлях через stepTo, який фільтрує ВСІХ сиблінгів (siblings.filter(...).indexOf), і одразу виконує XPath через findBySelector, де крок child знову фільтрує всіх дітей батька (children[parent].filter). Для текстових вузлів те саме робить pathFromIdAbove. На дереві, де під одним вузлом з id лежать n безіменних сиблінгів, це ~2·n² операцій на дерево. Обмеження 10 МиБ (040) дозволяє сотні тисяч вузлів (`<n bounds="[0,0][1,2]"/>` ≈ 25 байт).

Доказ.

```
Підтверджено на стенді (repro-005): дерево з 10 000 безіменних сусідніх вузлів (826 КБ XML, 110 КБ у gzip+base64) — GET /test-runs/:id/structure 6,8 с, паралельний GET /users/me чекав 5,8 с (event loop заблоковано для всіх команд). Складність квадратична: 30 000 вузлів — орієнтовно хвилина на кожне відкриття знімка. selector-variants.ts:86-90: `const siblings = childrenOf(tree)[node.parent]; const position = siblings.filter((index) => tree.nodes[index].className === node.className).indexOf(node.index) + 1;` викликається для кожного вузла з ui-tree.service.ts:162-163 (`selectorVariants(tree, node)`) і 181 (`textFields` → pathFromIdAbove). xpath-selector.ts:177: `const matching = children[parent].filter((child) => fits(tree.nodes[child], step.test));`. Офлайн-симуляція саме цих двох циклів (node 22, scratchpad sim/wide.js): 10 000 сиблінгів — 1,96 с, 30 000 — 28,9 с (≈1,7 МБ XML); 100 000 сиблінгів екстраполюється в ~5 хв; викликається двічі (tree і baselineTree). Живий запит не виконувався.
```

Вплив. Будь-хто з правом завантаження (API-ключ CI, editor) один раз надсилає знімок із широким деревом; кожне відкриття цього знімка будь-яким переглядачем (і відкриття знімків, для яких дерево стало еталоном) зупиняє API для всіх команд на десятки секунд–хвилини. Повторні відкриття множать ефект.

Рекомендація. Не перевіряти варіант повним XPath (позиція й кількість збігів уже відомі з кешу позицій) або обмежити кількість контекстів якоря; винести /structure у воркер з тайм-аутом; тест на 10 000 рядків з однаковим resource-id.

VIS-SEC-058 — Новий «лінійний» рушій виразів (RE2JS) дуже дорогий: 46–130 МіБ і ~1 с на текст для одного виразу, кеш на 500 виразів; guest може покласти API через selector-matches

High · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)` · **CWE:** CWE-770 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M · **Пов'язано з:** VIS-SEC-015

Де: `backend/src/ui-tree/linear-regex.ts:330-358; backend/src/ui-tree/comparison-inputs.ts:111-121,135-158,160-176; backend/src/test-runs/test-runs.controller.ts:235-242; backend/src/test-runs/test-runs.service.ts:551-565`

Опис. 051 замінив RegExp на RE2JS 2.9.1. Він справді не має експоненти: (a+)+$ на 2000 символах — 14 мс, вкладені повтори понад 1000 RE2 відхиляє ('unsupported', відмова не мовчазна). Але вартість і компіляції, і кожного test() пропорційна розміру скомпільованої програми. Вираз `(?:x*x*…x*){1000}` (кожен x* усередині {1000} множиться на 1000) у межах 300 символів дає програму із сотень тисяч інструкцій. RE2JS не має ліміту на розмір програми, як RE2 C++ (max_mem): відмова настає лише десь між 800 і 1800 символами. Скомпільований вираз лягає в кеш `compiled` (до 500 записів, linear-regex.ts:330-358) і тримає 46 МіБ (300 символів) або ~130 МіБ (~820 символів). Кожен test() на короткому тексті займає 0,15–1 с. Це порушує REQ-051-11 (≤50 мс на вираз і текст до 2000 символів). Досяжність: (1) GET /test-runs/:id/selector-matches приймає `selector` із query без ліміту довжини й типу. Маршрут під MixedGuard + @ProjectFrom, тож доступний guest-учаснику проєкту чи API-ключу. selectorProblem() → compileLinear() компілює й кешує вираз, findBySelector() виконує його над кожним текстовим вузлом дерева й еталона в головному потоці. (2) editor зберігає правило text:/…/ до 300 символів у проєкті. resolveRules виконує його над кожним вузлом дерева й еталона під час кожного порівняння кожного знімка (ui-tree.service.ts:619, головний потік).

Доказ.

```
Скрипти й виводи: /tmp/claude-0/-home-user-vision/82ab6902-a0af-5888-a773-e95df8c0504a/scratchpad/agent-C/fuzz3.ts, fuzz3-out.txt, fuzz5.ts, fuzz6.ts, fuzz6-out.txt, fuzz7-out.txt, mem.ts, mem-out.txt, mem300-out.txt.
- `(?:x*x*x*x*x*x*x*x*x*x*){1000}[^x]` (34 символи) на тексті з 2000 символів: test 1418 мс.
- Селектор рівно 300 символів `text:/(?:x*x138){1000}[qz]$/`: selectorProblem → null (приймається). findBySelector на дереві з 50 вузлів «Баланс 1 234,56 ₴ N»: 7935 мс.
- Селектор 1100 символів (selector-matches не обмежує довжину), 5 вузлів: 4793 мс.
- Пам'ять: 5 різних виразів по 300 символів у кеші — +230 МіБ heap; 5 виразів по ~820 символів — +659 МіБ (≈130 МіБ кожен).
- Код: linear-regex.ts:348-349 `RE2JS.compile(translated, …)` / `pattern.matcher(...).find()`; :357-358 `if (compiled.size >= CACHE_SIZE) … compiled.set(key, result)` з CACHE_SIZE = 500.
- test-runs.controller.ts:240-242 `@UseGuards(MixedGuard) selectorMatches(@Param('id') id, @Query('selector') selector: string)` — без DTO і MaxLength.
Наскрізно через API на стенді (raw/repro-009-re2-cost.txt): guest-учасник проєкту, один GET selector-matches із селектором 296 символів — 200 за 7526 мс, event loop заблоковано до 7034 мс, heap +46 МіБ, що лишаються в кеші.
```

Вплив. Один guest або власник API-ключа будь-якого проєкту кількома десятками GET-запитів з різними виразами доводить heap API до OOM: процес падає для всіх команд. Кожен запит до того ж блокує event loop на секунди–хвилини. Одне правило editor-а в проєкті додає ~8 с головного потоку на дерево до кожного порівняння (знімок + еталон ≈16 с), і весь цей час API не відповідає нікому. Це та сама загроза, яку 051 мав закрити (US-4, SC-051-04).

Оцінка. CVSS 6.5; severity піднято до High для узгодженості з VIS-SEC-043 (той самий клас: DoS усіх команд від будь-якого учасника проєкту) і через OOM-падіння процесу.

Рекомендація. Обмежити розмір програми: перед компіляцією оцінити кількість інструкцій, перемноживши лічильники вкладених повторів на розмір їхнього тіла, і відхиляти понад ~10–20 тис. Або в translate() відхиляти повтор {n>…} над групою з квантифікаторами чи сумарний добуток повторів > 1000. Кеш обмежити за розміром програми або байтами, а не за кількістю. Для selector-matches додати DTO з @IsString @MaxLength(300) (як у element-rule.dto.ts:13) і не кешувати вираз, що не був збережений. Додати тест REQ-051-11 саме з виразами (x*){1000} проти таймера 50 мс. Резолв правил винести у воркер з тайм-аутом (як a11y).

VIS-SEC-004 — Обчислювальна DoS: аналіз доступності UI-дерева блокує event loop для всіх команд

Medium · **Стан:** частково · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:N/A:H (7.7)` · **CWE:** CWE-400 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M

Де: `['backend/src/test-runs/test-runs.service.ts:144', 'backend/src/ui-tree/ui-tree.service.ts:352', 'backend/src/a11y/a11y.rules.ts:238-241', 'backend/src/a11y/contrast.ts:26,49']`

Перевірка 2026-11-08: backend/src/ui-tree/ui-tree.service.ts:262-263 (findViolations іде в diffWorkerPool.run({kind:'a11y'}) з тайм-аутом 60 с, diff-worker-pool.ts:19,177); ui-tree.service.ts:336-349 (decode() → staticService.getImage → readPng → PNG.sync.read) досі в головному потоці — Правила доступності у воркері. Декодування до трьох PNG (≤40 Мпкс) і розбір дерева лишились у головному потоці (VIS-SEC-046). Після 3 падінь воркера все знову йде в головний потік (VIS-SEC-052).

Опис. На шляху прийому `analyzeUiTree` синхронно викликає `findViolations`, яка для кожного текстового вузла рахує контраст, семплюючи до MAX_SAMPLES=40000 пікселів (contrast.ts). Усе синхронно в головному потоці, без воркера. Дерево обмежене лише 10 МиБ після розпакування — це десятки тисяч вузлів.

Доказ.

```
Статично підтверджено (audit/raw/repro-003-ignore-areas.txt): intake→analyzeUiTree(test-runs.service.ts:144)→findViolations (синхронно). Вартість ~O(N_textual×40000). Поправка до оцінки агента: checkDuplicateLabels лінійний, не квадратичний. Живий payload не запускався.
```

Вплив. Один невеликий після стиснення запит від CI будь-якої команди зупиняє API для всіх команд інстансу на секунди-десятки секунд (event loop зайнятий). Поруч робочий VRT на тій самій машині — спільний CPU.

Рекомендація. Винести аналіз доступності у воркер-пул із тайм-аутом; обмежити кількість вузлів дерева і сумарну площу контрастних боксів; відхиляти дерева понад розумну кількість вузлів. Потребує специфікації (зміна поведінки прийому).

VIS-SEC-010 — Дефолтний адмін із паролем 123456: нижче політики, друкується в лог, без примусової зміни

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N (8.1)` · **CWE:** CWE-798 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** prisma · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-4

Де: `['backend/prisma/seed.ts:43', 'docker-compose.demo.yml:129']`

Перевірка 2026-11-08: backend/prisma/seed.ts:43 `process.env.DEFAULT_USER_PASSWORD || '123456'`; seed.ts:72-74 друкує пароль у лог; docker-compose.demo.yml:132 `${DEFAULT_USER_PASSWORD:-123456}` — Без змін: 6 символів < політики 12 (account.ts MIN_PASSWORD_LENGTH), пароль у stdout, примусової зміни немає.

Опис. На порожній базі seed створює admin@vision.local з паролем за замовчуванням (у demo — 123456), коротшим за політику 12 символів, і друкує факт у лог. Примусової зміни при першому вході немає (свідомо, рішення власника).

Доказ.

```
seed.ts:43; demo compose DEFAULT_USER_PASSWORD; лог seed на стенді друкує email і пароль.
```

Вплив. Якщо дефолт не змінено до першого доступу в мережу банку — повний адмін-доступ за відомим паролем.

Рекомендація. Генерувати випадковий пароль, не друкувати його, вимагати зміну при першому вході; demo-пароль не тягнути в поставку. Примусова зміна — потребує специфікації.

VIS-SEC-016 — AI_PROVIDER=none не вимикає зовнішній AI: особисті ключі Gemini і ключ проєкту діють далі

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N (7.7)` · **CWE:** CWE-16 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M

Де: `['backend/src/ai/ai.service.ts', 'docs/decisions/0020-personal-gemini-keys.md']`

Перевірка 2026-11-08: backend/src/ai/ai.service.ts:99-104: `const provider = personal ? 'gemini' : this.provider;` — з особистим ключем Gemini виклик іде і при AI_PROVIDER=none; від a3b6104 src/ai і src/compare/libs/vlm не змінювались (git log a3b6104..HEAD -- src/ai src/compare/libs/vlm порожній) — Вимикача зовнішнього AI на рівні інстансу немає.

Опис. Налаштування `AI_PROVIDER=none` стосується лише чернеток багів на вимогу, але не перекриває особисті ключі Gemini користувачів і ключ Gemini проєкту (VLM-рушій, VIS-SEC-003). Єдиного вимикача зовнішнього AI на рівні інстансу немає.

Доказ.

```
ai.service.ts (гілки провайдерів); ADR-0020.
```

Вплив. Адмін банку не може однією настройкою гарантувати, що жодні дані не підуть у Gemini.

Рекомендація. Жорсткий вимикач рівня інстансу, що перекриває і VLM-рушій, і особисті/проєктні ключі. Потребує специфікації.

VIS-SEC-017 — Адмін задає пароль будь-якому акаунту (зокрема SSO/LDAP) і діє його токеном Jira

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:N (6.0)` · **CWE:** CWE-269 · **OWASP:** A01:2021-Broken Access Control · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** M

Де: `['backend/src/users/users.controller.ts:189']`

Перевірка 2026-11-08: backend/src/users/users.service.ts:109-121 setPassword(id,...) без перевірки, що акаунт локальний; users.controller.ts:223-230 — 052 лише додав завершення сесій після пароля від адміна; адмін і далі ставить локальний пароль акаунту LDAP/SSO і входить ним (діє його токеном Jira).

Опис. PUT /users/:id/password дозволяє адміну поставити локальний пароль будь-якому акаунту, навіть створеному через SSO/LDAP, і увійти ним локально — діючи його особистим токеном Jira (обхід духу REQ-035-33 «лише особисті токени»).

Доказ.

```
users.controller.ts:189 (admin-only, без заборони для зовнішніх акаунтів).
```

Вплив. Адмін підміняє особу співробітника в Jira й журналі рішень; немає розділення обов'язків.

Рекомендація. Заборонити локальний пароль для SSO/LDAP-акаунтів; аудит-лог адмінських дій над акаунтами. Потребує специфікації.

VIS-SEC-019 — Dev-compose у дереві: Postgres на 0.0.0.0 з postgres/postgres, ім'я контейнера збігається з продом, моки LDAP/OIDC

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (8.0)` · **CWE:** CWE-1188 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** infra · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-9

Де: `['backend/docker-compose.yml:48,56', 'backend/docker-compose.ldap.yml', 'backend/docker-compose.oidc.yml']`

Перевірка 2026-11-08: backend/docker-compose.yml:48 `container_name: postgres`, :55-56 `"${POSTGRES_PORT}:5432"` (0.0.0.0); backend/docker-compose.ldap.yml:26-27 `"636:636"`, :8 LDAP_BIND_PASSWORD 'adminpass'; backend/docker-compose.oidc.yml:41-42 `'8090:8080'`, :10 'vision-secret' — Без змін з a3b6104. Redis у dev-compose лише expose (без публікації) — гаразд.

Опис. Dev-compose публікує Postgres `${POSTGRES_PORT}:5432` на всіх інтерфейсах з postgres/postgres і `container_name: postgres` — те саме ім'я, що в робочого VRT (ризик колізії на спільній машині, правило 1). Моки LDAP/OIDC із відомими секретами. Якщо dev-compose випадково потрапить у поставку — відкрита база.

Доказ.

```
backend/docker-compose.yml:48 (container_name: postgres), :56 (${POSTGRES_PORT}:5432).
```

Вплив. На спільній із продом машині запуск dev-compose може зіткнутися з контейнером VRT; відкритий Postgres із дефолтним паролем у мережі банку.

Рекомендація. Прив'язати порт до 127.0.0.1, унікальне ім'я контейнера, явно виключити dev-compose з поставки (вони лише для розробки); demo compose вже не публікує Postgres.

VIS-SEC-021 — Запобіжник агентів guard.py легко обходиться — правила 1–4 тримаються лише на дисципліні

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:H (7.3)` · **CWE:** CWE-807 · **OWASP:** A04:2021-Insecure Design · **Компонент:** ci · **Впевненість:** High · **Зусилля:** M

Де: `['.claude/hooks/guard.py']`

Перевірка 2026-11-08: .claude/hooks/guard.py:126-136 (перевірка лише args[:1]=='compose' після позиційних аргументів; немає розбору bash -c/eval, docker container stop, docker exec). Прямий виклик check(): `docker --context default compose -p vrt down -v` → ALLOW; `bash -c 'docker compose down'` → ALLOW; `docker container stop postgres` → ALLOW; `docker exec postgres dropdb vrt` → ALLOW; `git push --force vision main` → ALLOW; `docker -H unix:///var/run/docker.sock compose down` → ALLOW; `eval docker\ compose\ down` → ALLOW (скрипт scratchpad/agent-D/g3.py) — guard.py не змінювався з a3b6104. Як і раніше — зручність, не контроль безпеки.

Опис. Перевірка аргументів у guard.py обходиться: `docker --context … compose`, `bash -c '…'`, `docker container stop`, `docker exec … dropdb`, `git push --force vision main`. Хук — зручність, не бар'єр безпеки; непорушні правила 1–4 (захист робочого VRT і бази) тримаються на дисципліні агента.

Доказ.

```
Пряма перевірка функції check() (scratchpad/agent-D/g.py).
```

Вплив. На спільній із продом машині агент (або скомпрометований агентський процес) може зачепити робочий VRT попри хук.

Рекомендація. Не покладатися на guard.py як на контроль безпеки; ізолювати права Docker/БД агента на рівні ОС (окремий користувач, обмежений доступ до сокета Docker, read-only до прод-даних).

VIS-SEC-046 — Розбір XML, серіалізація дерева і декодування PNG при прийомі досі в головному потоці; глибина дерева дає ×200 підсилення

Medium · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)` · **CWE:** CWE-400 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** M · **Пов'язано з:** VIS-SEC-004

Де: `['backend/src/ui-tree/ui-tree.service.ts:276-298', 'backend/src/ui-tree/ui-tree.parser.ts:20-27', 'backend/src/ui-tree/ui-tree.parser.ts:166-190', 'backend/src/ui-tree/ui-tree.parser.ts:220-228', 'backend/src/ui-tree/ui-tree.service.ts:305-320', 'backend/src/ui-tree/ui-tree.service.ts:336-349', 'backend/src/static/hdd/hdd.service.ts:49-57', 'backend/src/ui-tree/ui-tree.intake.ts:20-38']`

Перевірка 2026-11-08: backend/src/ui-tree/ui-tree.service.ts:278-300: readUiTreeXml → parseUiTree (fast-xml-parser) → JSON.stringify → gzipSync синхронно в головному потоці; : 336-349 декодування PNG там само — Нове пом'якшення, якого попередня ітерація не врахувала: fast-xml-parser 5.12.0 має типове maxNestedTags=100 (node_modules/fast-xml-parser/src/xmlparser/OptionsBuilder.js:54), тож глибина дерева ≤100 і підсилення ×200 через шлях обмежене ~×100. Головний потік і відсутність ліміту кількості вузлів лишились.

Опис. Після 88669e6 у воркер пішли лише правила доступності. При кожному прийомі з деревом у головному потоці синхронно: fast-xml-parser.parse до 10 МиБ XML, рекурсивний walk, обчислення ключів `path:${node.path}`, JSON.stringify і gzipSync усього дерева (store), а в analyze — декодування до трьох PNG до 40 Мпкс (знімок, еталон, еталон іншого пристрою) через StaticService.getImage → readPng → PNG.sync.read, diffTrees і componentChanges. Ключ безіменного вузла містить повний шлях від кореня, тож розмір збереженого JSON росте як глибина², а ліміт 10 МиБ рахує вхід, а не результат. Захист від XXE/billion laughs працює: DOCTYPE/ENTITY відкидається (ui-tree.intake.ts:35), gunzip обмежено maxOutputLength (24), fast-xml-parser не тягне зовнішніх сутностей.

Доказ.

```
ui-tree.parser.ts:186: `path = `${parentPath}/${shortClass(node.className)}[${siblingIndex}]`;` i :222 `: `path:${node.path}``; ui-tree.service.ts:293-298 `gzipSync(JSON.stringify(tree))`. Офлайн-симуляція того самого walk/ключів/stringify/gzip (scratchpad sim/depth2.js, node 22): 12 ланцюжків глибиною 3000 (≈2,2 МБ XML, gzip ще менше) → walk+ключі 2,0 с, JSON 434 МБ за 2,6 с, gzip 1,8 с: разом 6,4 с блокування event loop і ~0,7 ГБ пам'яті на один запит без урахування самого fast-xml-parser. Глибше ≈3500 рекурсія walk падає з RangeError (перехоплюється в store:296 — дерево відкидається). Далі збережене дерево читає load() (JSON.parse 434 МБ у головному потоці, кеш до 300 дерев — ui-tree.service.ts:36,305-320) і копіює в воркер для a11y.
```

Вплив. Кілька паралельних завантажень на хвилину від одного CI-ключа тримають API недоступним для всіх команд; кеш дерев на 300 записів із деревами по сотні МБ веде до вичерпання heap і падіння процесу.

Рекомендація. Винести store (розбір, серіалізацію) і analyze повністю у воркер-пул із тайм-аутом; обмежити глибину (напр. 200) і кількість вузлів дерева при розборі; не тримати повний шлях у ключі (хеш шляху або індекс), обмежити розмір серіалізованого дерева; кеш дерев обмежувати за байтами. Декодування PNG для a11y робити у воркері, передавати буфер PNG, а не RGBA. Потребує специфікації (зміна поведінки прийому).

VIS-SEC-059 — Збереження профілю (PUT /users) відкриває новий вхід, а старий токен цього браузера лишається дійсним і після виходу

Medium · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N (5.9)` · **CWE:** CWE-613 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** VIS-SEC-009

Де: `backend/src/users/db/dbusers.service.ts:44; backend/src/users/ldap/ldapusers.service.ts:149; backend/src/users/oidc/oidcusers.service.ts:346; frontend/src/services/users.service.ts:78-83; backend/src/auth/auth.service.ts:78-84`

Опис. Усі три реалізації update() викликають authService.signToken(user), який створює НОВИЙ рядок Session з новим sid. Фронтенд після збереження профілю замінює токен у localStorage на новий. Попередній sid цього ж браузера ніхто не завершує (endedAt лишається null). Вихід надсилає лише новий токен, тож endSession() завершує лише новий sid; старий токен — той,

що міг бути скопійований раніше, — працює до свого exp. Це ламає REQ-052-01 / SC-052-01 («після виходу 100% запитів зі старим токеном — не авторизовано») для будь-кого, хто хоч раз зберіг профіль. Тест sessions.e2e-spec.ts цей сценарій не покриває.

Доказ.

```
dbusers.service.ts:35-45: `const user = await this.prismaService.user.update(...); const token = await
this.authService.signToken(user); return new UserLoginResponseDto(user, token);`
auth.service.ts:41-48: `const sid = randomUUID(); ... await this.prismaService.session.create({ data: { id:
sid, userId: user.id, ... } })`
frontend users.service.ts:78-83 (update): `.then(handleResponse).then((user) =>
{ setUserInLocalStorage(user); ...})` — токен замінено.
users.controller.ts:220 logout: `this.authService.sessionIdOf(request.headers.authorization)` — лише sid
нового токена.
Відтворено на стенді (raw/repro-008-sessions-profile-password.txt): після PUT /users і виходу новий токен →
401, старий → 200.
```

Вплив. Украдений до збереження профілю токен переживає вихід людини (і, якщо вона не змінює пароль, діє до кінця доби). Людина вважає, що вийшла. Також у режимі БД зміна email у профілі робить старі токени тимчасово «no-user», але не завершує їхні сесії: повернення попереднього email знову їх оживляє.

Примітка. Незалежно знайдено двома агентами (A, B).

Рекомендація. У update() не відкривати новий вхід: або повертати той самий токен (без нового sid), або підписувати новий токен із sid поточного запиту (sessionIdOf) і не створювати новий рядок; альтернатива — завершувати попередній sid у тій самій транзакції. Додати e2e: увійти → PUT /users → вийти → старий і новий токени дають 401.

VIS-SEC-060 — Зміна пароля, email і випуск API-ключа без повторної автентифікації: викрадений токен закріплюється й виганяє власника

Medium · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:N (6.8)` · **CWE:** CWE-620 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M

Де: `backend/src/users/users.controller.ts:196-210 (PUT /users/password), 183-190 (GET /users/newApiKey), 231-238 (PUT /users); backend/src/users/users.service.ts:83-98`

Опис. PUT /users/password приймає лише новий пароль: поточний не перевіряється. Після 052 зміна пароля завершує всі ІНШІ входи й лишає вхід того, хто змінив (REQ-052-02). Отже той, хто заволодів токеном (XSS, скопійований localStorage, чужий ПК), міняє пароль локального акаунта, лишається увійденим, а власник втрачає всі сесії і не може увійти. Додатково GET /users/newApiKey з тим самим токеном видає безстроковий API-ключ, який за REQ-052-05 переживає вихід і зміну пароля, а в режимі БД PUT /users без пароля змінює email (логін).

Доказ.

```
users.controller.ts:200-210: `changePassword(@CurrentUser() user, @Body('password') password, @Req()
request) { return this.usersService.changePassword(user, password, this.authService.sessionIdOf(...)) }` —
жодного currentPassword.
users.service.ts:93-97: `if (changed && (!ldap || local)) { await this.authService.endSessions(user.id,
sessionId); await this.eventsGateway.endSessions(user.id, { keep: sessionId }); }`
api.guard.ts:13-19 — ключ без строку дії; spec 052 REQ-052-05.
Відтворено на стенді (raw/repro-008-sessions-profile-password.txt): PUT /users/password без поточного пароля
```

```
→ 200; токен власника → 401, викрадений → 200; вхід власника своїм паролем → 400; новий API-ключ викраденим токеном → 200.
```

Вплив. Короткочасна крадіжка токена (доба) перетворюється на постійне захоплення локального акаунта (зокрема адміна) з виселенням власника; API-ключ дає доступ після будь-яких дій власника, крім ручної ротації ключа чи видалення акаунта адміном.

Примітка. Незалежно знайдено двома агентами (A — Medium, B — Low); зведено як Medium: захоплення локального адміна з виселенням власника.

Рекомендація. Вимагати поточний пароль для PUT /users/password і зміни email (у режимі БД) та, бажано, для випуску API-ключа; при зміні пароля людиною також інвалідовувати API-ключ або показувати попередження. Потребує невеликої специфікації (зміна API, фронтенд-форма).

VIS-SEC-061 — Області ігнорування, які сервер виводить із дерева (системні панелі, правила), обходять ліміт 20 і займають воркер порівняння до тайм-ауту

Medium · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L (4.3)` · **CWE:** CWE-770 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** VIS-SEC-005

Де: `backend/src/ui-tree/comparison-inputs.ts:296-305; backend/src/ui-tree/ui-tree.service.ts:614-633; backend/src/test-runs/test-runs.service.ts:453-455; backend/src/compare/utils/index.ts:14-28`

Опис. REQ-051-09 обмежує 20 областями лише те, що надходить у запиті; області з дерева «додає сервер» і до ліміту не рахуються (граничний випадок у spec 051). Проте дерево надсилає той самий клієнт. findSystemBars бере межі кожного вузла з resource-id android:id/statusBarBackground або navigationBarBackground без обмеження кількості. Прапорець excludeSystemBars задає сам клієнт у запиті (CreateTestRequestDto.excludeSystemBars → uiTreeOptions, test-runs.service.ts:453-455; пріоритет над налаштуванням проєкту, ui-tree.service.ts:615-616). Отже, SDK-клієнт з API-ключем надсилає дерево з N вузлами statusBarBackground на весь екран, і порівняння отримує N повноекранних областей. applyIgnoreAreas у воркері проходить кожну з них попіксельно. Так само працюють правила ignore за class/text проєкту (resolveRules → resolved.ignore) і moves (compensateMoves → shiftBaseline).

Доказ.

```
Скрипт /tmp/claude-0/-home-user-vision/82ab6902-a0af-5888-a773-e95df8c0504a/scratchpad/agent-C/ign.ts, вивід ign-out.txt: одне правило ignore `class:android.widget.ProgressBar` на дереві з 200 повноекранних вузлів дає 200 областей; applyIgnoreAreas на 1080×2400 — 1177 мс. Лінійно ≈ 10 000 вузлів (≈1,5 МБ XML, у межах 10 МиБ) ≈ 60 с, тобто тайм-аут воркера (diff-worker-pool.ts:19). Код: comparison-inputs.ts:300-303 `tree.nodes.filter((node) => STATUS_BAR_IDS.includes(node.resourceId) && …).map((node) => node.bounds)` — без ліміту; ui-tree.service.ts:627-630 `ignoreAreas: [...[...systemBars, ...resolved.ignore].map(clamp)…]`.
```

Вплив. Кожен такий знімок тримає воркер порівняння до 60 с. Вісім паралельних завантажень (типовий розмір пулу) зупиняють порівняння для всіх команд, поки атака триває. Це той самий сценарій US-3, який 051 мав закрити. Для проєкту з рушієм VLM ті самі області застосовуються ще й у головному потоці (vlm.service.ts:85-86).

Рекомендація. Обмежувати кількість областей після злиття (системні панелі + правила + запит): наприклад, брати не більше 1–2 системних панелей кожного id, об'єднувати області, що перетинаються, і ставити жорстку межу (≈200) з позначкою в результаті. Або застосовувати області маскою за один прохід (позначити прямокутники в бітмапі й пройти зображення один раз), щоб вартість не залежала від їхньої кількості.

VIS-SEC-022 — Підписане посилання на знімок безстрокове й не прив'язане до користувача

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-200 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-5

Де: `['backend/src/static/file-links.ts:5-9,29', 'backend/src/static/static.controller.ts:16']`

Перевірка 2026-11-08: backend/src/static/file-links.ts:29 signFile = mac(key, `file\n${name}`) — без строку дії й прив'язки до користувача; static.controller.ts:76-93 — Свідоме рішення REQ-043-03, але для банку ризик лишається: посилання з Jira чи логу діє вічно. Посилання звіту гине разом із ним (REQ-043-06), це працює.

Рекомендація. Додати строк дії й прив'язку до користувача/сесії; переоцінити REQ-043-03 для банку.

VIS-SEC-023 — Підробка записів у лозі через назву перевірки

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-117 · **OWASP:** A09:2021-Security Logging and Monitoring Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/ui-tree/ui-tree.service.ts:269-271']`

Перевірка 2026-11-08: backend/src/ui-tree/ui-tree.service.ts:281-283 `this.logger.warn(` UI tree of "${check.name ?? '?'}" in project ${check.projectId ?? '?'} left out (${intake.left}): ${intake.detail}`)` — без екранування \r\n; також :302, :322 `${error}` — Рядки змістилися (було 269-271), логіка та сама.

Рекомендація. Екранувати переноси в логованих значеннях; структуроване логування.

VIS-SEC-024 — Шифрування секретів без AAD, KDF = один SHA-256; приймається вже зашифроване значення

Low · **Стан:** частково · **CVSS:** `n/a (Low)` · **CWE:** CWE-327 · **OWASP:** A02:2021-Cryptographic Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-3

Де: `['backend/src/shared/secret-box.ts:12-23,34']`

Перевірка 2026-11-08: backend/src/shared/server-secrets.ts:30-54 + main.ts:32-39: VISION_SECRET_KEY ≥32 символи, не з прикладів, ≠ JWT_SECRET (051); але secret-box.ts:14 KDF = один SHA-256, :21 AES-GCM без AAD; projects/project-secrets.ts:38 `!isEncrypted(next.apiKey)` — значення enc:v1: з клієнта зберігається як є — Слабкий ключ тепер не пройде старт. AAD і прийом готового шифротексту не змінились (шифротекст наружу не віддається, тож підставити можна лише з дампу).

Рекомендація. Додати AAD (контекст поля), вимагати достатню ентропію ключа, не приймати enc:-значення ззовні.

VIS-SEC-025 — Токени публічних звітів і токени Google Drive зберігаються відкрито

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-312 · **OWASP:** A02:2021-Cryptographic Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/prisma/schema.prisma (ReportLink.token)', 'backend/src/backups/drive-auth.ts']`

Перевірка 2026-11-08: backend/prisma/schema.prisma:399 `token String @unique` (відкритим текстом); reports.service.ts:104 і static.controller.ts:86 шукають за відкритим токеном; backups/drive-auth.ts:102-110 токени Drive у файлі rclone відкритим текстом — ADR-0022 (публічні звіти) прийнято, але зберігання хешу токена не зроблено.

Рекомендація. Зберігати лише хеш токена звіту (перевірка за хешем); токени Drive — через secret-box.

VIS-SEC-026 — AI-ендпоінти на вимогу доступні ролі guest (платні виклики й дані проєкту)

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-285 · **OWASP:** A01:2021-Broken Access Control · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/ai']`

Перевірка 2026-11-08: backend/src/ai/ai.controller.ts:95-120 — POST /ai/group-summary, /ai/a11y-labels, /ai/bug-draft мають лише @UseGuards(JwtAuthGuard), без RoleGuard/@Roles (файл не змінювався з 7b13af1). backend/src/reports/reports.controller.ts:43-49 — GET /builds/:id/report-links лише JwtAuthGuard: guest отримує token кожного чинного публічного звіту, хоча створювати їх може лише editor/admin (REQ-031-69, рядок 30). — Ізоляція проєктів на цих маршрутах працює (@ProjectFrom на buildId/testRunId). Лишається: guest запускає LLM-виклики на ключі проєкту чи інстансу і може роздати назовні вже створене посилання на звіт.

Рекомендація. Обмежити щонайменше editor; окремий ліміт на guest.

VIS-SEC-027 — У логи потрапляють відповіді моделі (тексти екранів) і URL із підписами/токенами звітів

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-532 · **OWASP:** A09:2021-Security Logging and Monitoring Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/ai', 'backend/src (Logger)']`

Перевірка 2026-11-08: backend/src/compare/libs/vlm/vlm.service.ts:135 `this.logger.debug(` VLM response content: `${content})` (main.ts не обмежує рівні логера, тож debug у Nest увімкнено); ai/ai.service.ts:144 пише 200 символів відповіді моделі; http-exception.filter.ts:20-21 логує request.url з query (?sig=…, ?selector=…, ?url=…) і шлях public/reports/:token на кожній 4xx/5xx — Без змін від попередньої ітерації.

Рекомендація. Не логувати тіла відповідей моделі й повні підписані URL; редагувати токени.

VIS-SEC-028 — CLI пересилає заголовок apiKey при редиректі на інший хост/схему

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-200 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** cli · **Впевненість:** High · **Зусилля:** S

Де: `['frontend/public/cli/vision.py:102,212']`

Перевірка 2026-11-08: frontend/public/cli/vision.py:95-102 urllib.request.urlopen з заголовком apiKey; стандартний HTTPRedirectHandler.redirect_request (перевірено на локальному Python) копіює всі заголовки, крім content-*, на новий URL будь-якого хоста й схеми; CLI не вимагає https (:513 приклад http://vision:14200) — Без змін.

Рекомендація. Заборонити редиректи або не пересилати apiKey на інший origin; вимагати https.

VIS-SEC-029 — test:pw:install вимикає перевірку TLS під час завантаження браузера

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-295 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** frontend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-8

Де: `['frontend/package.json:48']`

Перевірка 2026-11-08: frontend/package.json:48 `"test:pw:install": "NODE_TLS_REJECT_UNAUTHORIZED=0 npx playwright install --with-deps chromium"`; викликається з test:pw:local і test:pw:debug (:49-50) — Лише dev.

Рекомендація. Прибрати прапорець; у контурі — корпоративний CA через NODE_EXTRA_CA_CERTS.

VIS-SEC-030 — CI: actions за тегами, не за SHA; не обмежено permissions GITHUB_TOKEN

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-1357 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** ci · **Впевненість:** High · **Зусилля:** S

Де: `['.github/workflows/docs.yml:12,13']`

Перевірка 2026-11-08: .github/workflows/docs.yml:12-13 `actions/checkout@v4`, `actions/setup-python@v5`; немає `permissions:` ні на рівні workflow, ні job — Єдиний workflow; тригер pull_request (не pull_request_target) — ризик обмежений.

Рекомендація. Закріпити actions за SHA; додати `permissions: contents: read`.

VIS-SEC-031 — Gradle wrapper 6.1 без distributionSha256Sum і без перевіреної контрольної суми jar

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-494 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** sdk-java · **Впевненість:** Medium · **Зусилля:** S

Де: `['sdk-java/gradle/wrapper/gradle-wrapper.properties', 'sdk-java/gradle/wrapper/gradle-wrapper.jar']`

Перевірка 2026-11-08: sdk-java/gradle/wrapper/gradle-wrapper.properties: distributionUrl=gradle-6.1-all.zip без distributionSha256Sum; sha256(gradle-wrapper.jar)=96f793a18e056c23ffeec67c1f3bb8eccff5a4a407fc9ceac183527e7eedf4b6 (не звірено з офіційною — мережа); build.gradle: shadow 5.2.0, lombok-plugin 5.1.1 — Без змін.

Рекомендація. Додати distributionSha256Sum; звірити jar з офіційним для версії; оновити застарілі плагіни (shadow 5.2.0, lombok 5.1.1).

VIS-SEC-032 — Prompt injection: тексти екрана й назви перевірок у промптах; вердикт VLM сам ставить статус

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-94 · **OWASP:** A03:2021-Injection · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** S

Де: `['backend/src/ai/gemini.service.ts', 'backend/src/compare/libs/vlm/vlm.service.ts:98-99']`

Перевірка 2026-11-08: src/ai і src/compare/libs/vlm не змінювались від a3b6104; vlm.service.ts:99-103 `if (pass) result.status = TestStatus.ok` — Вердикт моделі досі сам ставить ok. Сам VLM при прийомі виключено (VIS-SEC-003), а injection у вердикт — ні.

Рекомендація. Не дозволяти вердикту моделі самому затверджувати; розділяти інструкції й дані в промпті; екранувати.

VIS-SEC-033 — Енумерація користувачів за часом відповіді входу

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-203 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/users']`

Перевірка 2026-11-08: backend/src/users/db/dbusers.service.ts:59-66 — для невідомого email виняток до bcrypt compare; немає фіктивного bcrypt — Час відповіді й далі відрізняє наявний акаунт (~bcrypt cost 10) від невідомого.

Рекомендація. Виконувати фіктивний bcrypt для невідомих email (константний час).

VIS-SEC-034 — Ліміт входу лише за email: password spraying і блокування чужих акаунтів

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-307 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-6

Де: `['backend/src/users']`

Перевірка 2026-11-08: backend/src/users/sign-in-limit.ts:17,28-39 — ключ лише email; users.service.ts:123-139 — Ліміту за IP немає; анонім блокує вхід відомого email на 15 хв 5 запитами.

Рекомендація. Додати ліміт за IP (з коректним trust proxy) поряд з лімітом за акаунтом; не блокувати легітимного користувача.

VIS-SEC-035 — Звільнені з каталогу/IdP зберігають доступ через API-ключ (безстроковий, без перевірки каталогу)

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-613 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/users']`

Перевірка 2026-11-08: backend/src/auth/guards/api.guard.ts:11-21 — лише findUnique за хешем, без строку дії і стану акаунта; REQ-052-05 свідомо лишає ключ живим після виходу/зміни пароля — Лише частина про API-ключ: строку дії, перевірки каталогу/IdP, автоматичної ротації немає. Ентропія (140 біт, randomBytes) і зберігання (SHA-256) — гаразд; ротація вручну через GET /users/newApiKey замінює хеш. | Severity знижено з Medium до Low: частина про локальний вхід — це VIS-SEC-044, відкладена рішенням власника; лишається API-ключ без строку дії й перевірки каталогу.

Рекомендація. Перевіряти стан акаунта при кожному запиті; строк дії й ротація ключів.

VIS-SEC-036 — PUT /users записує email без нормалізації й перевірки дублікатів в іншому регістрі

Low · **Стан:** частково · **CVSS:** `n/a (Low)` · **CWE:** CWE-706 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/users/dbusers.service.ts:35']`

Перевірка 2026-11-08: backend/src/users/db/dbusers.service.ts:35-41 `email: userDto.email` без accountEmail(); унікальний індекс email чутливий до регістру — createAccount (users.service.ts:53-60), LDAP і OIDC нормалізують; режим БД у PUT /users — ні: можна поставити `BOB@bank` поряд з `bob@bank`.

Рекомендація. Нормалізувати email (lowercase) і забезпечити унікальність без урахування регістру (додавальний індекс).

VIS-SEC-037 — Скрипти запускають образи за тегом з обліковими даними проду і --network host

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-494 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** scripts · **Впевненість:** Medium · **Зусилля:** S

Де: `['scripts/demo-clone-from-prod.sh', 'scripts/lib/vision-backup.sh']`

Перевірка 2026-11-08: scripts/lib/vision-safety.sh:96 `docker run --rm --network host postgres:16-alpine psql -X "$PROD_DATABASE_URL"` (тег без digest, креденшали проду в argv → видно в ps); scripts/demo-clone-from-prod.sh:139 те саме для pg_dump; scripts/lib/vision-backup.sh:146 `VB_RCLONE_IMAGE="rclone/rclone:1.68.2"` без digest; scripts/control-scenario.sh:82 postgres:16-alpine без digest — У compose образ postgres закріплений digest (docker-compose.demo.yml:140), у скриптах — ні.

Рекомендація. Закріпити образи за digest; мінімізувати мережу; не передавати прод-креденшали у скрипти demo.

VIS-SEC-038 — vision-safety.sh: .env.demo виконується як shell і може перевизначити закріплені VISION_*

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-94 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** scripts · **Впевненість:** Medium · **Зусилля:** S

Де: `['scripts/lib/vision-safety.sh', 'scripts/lib/vision-backup.sh']`

Перевірка 2026-11-08: scripts/lib/vision-safety.sh:28-34 `set -a; . "$VISION_ENV_FILE"; set +a` — виконується як shell після встановлення VISION_PROJECT/VISION_COMPOSE_FILE (:13-14), тож рядок `VISION_PROJECT=vrt` у .env.demo перенаправляє vision_compose і vision_guard на інший compose-проєкт; VISION_ENV_FILE (:15) береться з оточення — Файл без shebang (бібліотека). vision-convert-env.py ставить 600 на .env.demo, але vision.sh права файла не перевіряє.

Рекомендація. Парсити .env без виконання (grep/export лише KEY=VALUE); права 600 на .env.

VIS-SEC-042 — Тестові фікстури містять фотографію реальних людей (успадковано від upstream)

Low · **Стан:** не виправлено · **CVSS:** `n/a (Low)` · **CWE:** CWE-359 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** L-2

Де: `['backend/test/image.png', 'backend/test/image_edited.png', 'backend/test/image_edited_2.png']`

Перевірка 2026-11-08: backend/test/image.png (sha256 e2a7df17…c245), image_edited.png, image_edited_2.png — переглянуто: фото реальних упізнаваних людей; файли не змінювались з a3b6104

Рекомендація. Замінити синтетичними зображеннями (як frontend/integration_tests/images/generate.mjs) або нейтральними патернами; переоцінити для банку.

VIS-SEC-049 — Лічильник спроб входу: квадратична CPU-DoS і необмежене зростання пам'яті від анонімних запитів

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H (6.5)` · **CWE:** CWE-407 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/users/sign-in-limit.ts:291-313', 'backend/src/users/users.service.ts:100-117']`

Перевірка 2026-11-08: backend/src/users/sign-in-limit.ts:46-50 forgetOld() O(n) на кожну невдачу при size ≥ 10000; closedUntil (рядок 13, 33) чиститься лише при перевірці того самого email — Код не змінювався з попередньої ітерації.

Рекомендація. Обмежити розмір обох мап (LRU/кільцевий буфер з жорсткою межею), чистити closedUntil разом із failures, прибирати застарілі записи амортизовано (за таймером раз на хвилину, а не на кожен запит). Додатково — грубий ліміт частоти /users/login на з'єднання/IP за коректним trust proxy (поза 045 — потребує рішення).

VIS-SEC-050 — Фільтр LDAP: String.replaceAll трактує $', $` і $& з email як шаблони заміни — обхід екранування

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N (3.7)` · **CWE:** CWE-90 · **OWASP:** A03:2021-Injection · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/users/ldap/ldapusers.service.ts:71', 'backend/src/users/ldap/ldapusers.service.ts:83']`

Перевірка 2026-11-08: backend/src/users/ldap/ldapusers.service.ts:83 `usersSearchFilter.replaceAll('{{email}}', email)` — рядок-заміна з $' $$& — Без змін; виправлення — функція-заміна() => email`.

Рекомендація. Підставляти функцією: `replaceAll('{{email}}', () => email)`; додатково відхиляти email з `$` до пошуку; тест на `$'`/`$``.

VIS-SEC-051 — LDAP: вимкнення перевірки TLS діє на весь клієнт без попередження, ldap:// без TLS приймається мовчки

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N (4.8)` · **CWE:** CWE-295 · **OWASP:** A02:2021-Cryptographic Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** S-12

Де: `['backend/src/users/ldap/ldapusers.service.ts:44-51', 'docker-compose.demo.yml:87,95']`

Перевірка 2026-11-08: backend/src/users/ldap/ldapusers.service.ts:44-51 rejectUnauthorized: !tlsNoVerify без попередження; ldap:// приймається мовчки — Для OIDC попередження є (oidcusers.service.ts:82-84), для LDAP — ні.

Рекомендація. Попередження в лог при NO_VERIFY і ldap://; у прод-профілі — відмова стартувати без ldaps:// або StartTLS; описати в DEPLOY.md.

VIS-SEC-052 — Три падіння воркера поспіль (зокрема повтори одного завдання) назавжди переводять пул порівняння в головний потік без тайм-ауту

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:N/A:H (5.3)` · **CWE:** CWE-636 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** Low · **Зусилля:** S · **Пов'язано з:** VIS-SEC-005

Де: `['backend/src/compare/diff-worker-pool.ts:108-157', 'backend/src/compare/diff-worker-pool.ts:159-170', 'backend/src/compare/diff-worker-pool.ts:62-64']`

Перевірка 2026-11-08: backend/src/compare/diff-worker-pool.ts:121-157 (spawnFailures += 1 за будь-яке падіння, зокрема повтор того самого завдання; ≥3 → inline=true назавжди) і :160-170 drainQueueInline; файл не змінювався від a3b6104 — Тайм-аут (timeOut, :185-196) правильно не рахується як spawn failure. Решта як у попередній ітерації.

Рекомендація. Рахувати spawn failures лише для воркерів, що впали до першого завдання (або під час старту); завдання, що впало двічі, відхиляти без зміни режиму пулу; не переходити на головний потік автоматично (краще відмовляти з помилкою), або повертатись до воркерів за часом; задати resourceLimits воркерам.

VIS-SEC-053 — XPath-селектори правил з осями // і ancestor мають необмежену складність і виконуються в головному потоці

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:N/A:H (4.9)` · **CWE:** CWE-407 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** S

Де: `['backend/src/ui-tree/xpath-selector.ts:160-195', 'backend/src/ui-tree/comparison-inputs.ts:152-159', 'backend/src/ui-tree/comparison-inputs.ts:206-227', 'backend/src/test-runs/dto/element-rule.dto.ts:13']`

Перевірка 2026-11-08: xpath-selector.ts:206-237: descendantsOrSelf для кожного вузла контексту, ліміту кількості кроків немає (parseXpath :76-107). Вимір (scratchpad/agent-C/xpath1.ts, xpath1-out.txt): дерево 19 096 вузлів глибиною 95 (межа fast-xml-parser 100), селектор 292 символи `xpath://*[@resource-id='app:id/root']` + `//*`×85 → findBySelector 2070 мс на одне дерево. resolveRules робить це для дерева й еталона на кожне порівняння (ui-tree.service.ts:619, головний потік). GET /test-runs/:id/selector-matches приймає селектор будь-якої довжини (test-runs.controller.ts:235-242, без DTO/MaxLength) від будь-кого з доступом до проєкту, зокрема guest і API-ключа (MixedGuard) — VIS-SEC-043 змінив лише крок child; вартість осей // і ancestor та кількість кроків не обмежені.

Рекомендація. Обмежити кількість кроків (напр. ≤ 8) і кількість descendant-кроків (≤ 2); рахувати descendant один раз на крок (позначка відвіданих), обмежити розмір контексту; виконувати резолв правил у воркері.

VIS-SEC-054 — Тексти з UI-дерева і назва перевірки потрапляють у wiki-розмітку Jira без екранування

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N (3.4)` · **CWE:** CWE-74 · **OWASP:** A03:2021-Injection · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** S

Де: `['backend/src/a11y/jira-bug.ts:43', 'backend/src/a11y/jira-bug.ts:85-110', 'backend/src/visual-bugs/visual-bug.ts:64-94', 'backend/src/ui-tree/ui-tree.parser.ts:273-280']`

Перевірка 2026-11-08: backend/src/a11y/jira-bug.ts:43 `const escapeWiki = (value) => value.replace(/\|/g, '/').replace(/[{}]/g, '')` застосовується лише до рядків таблиці (:97); `# ${v.message}` (:86), `* ${key}: ${value}` (:89) і backend/src/visual-bugs/visual-bug.ts:92 `Перевірка: ${check}` вставляються як є. Опис іде як wiki-розмітка через /rest/api/2/issue (jira.client.ts:278). git diff a3b6104..HEAD цих файлів не торкається. — JQL і шляхи REST і далі екрановані. jiraKey перевіряється регулярним виразом /^[A-Za-z][A-Za-z0-9_]*-\d+$/ (a11y-issue.dto.ts:4, visual-bug.dto.ts:15), тому `key = "${jiraKey}"` у a11y-issues.service.ts:546 не дає JQL-ін'єкції.

Рекомендація. Екранувати спецсимволи wiki-розмітки ([ ] ! { } | * _ # h1.) у всіх рядках з даних (або загортати у {noformat}); для Jira Cloud — ADF з текстовими вузлами.

VIS-SEC-055 — VLM-вердикт при прийомі запускає автоапрув і множить виклики Gemini без ліміту

Low · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N (4.2)` · **CWE:** CWE-1039 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/test-runs/test-runs.service.ts:770-850', 'backend/src/compare/libs/vlm/vlm.service.ts:57-104', 'backend/src/test-runs/test-runs.service.ts:661-678']`

Перевірка 2026-11-08: backend/src/compare/libs/vlm/vlm.service.ts:57-63 (примусово saveDiffAsFile:true), :99-103 вердикт → ok; test-runs.service.ts:822-850 tryAutoApproveByNewBaselines цикл getDiff по approved runs; :893 MAX_RECOMPARE_AFTER_RULE=200; файли не змінювались по суті — Без змін. Окремо від VIS-SEC-003: тут автоапрув і множення платних викликів, а не сам факт VLM.

Рекомендація. Не дозволяти вердикту VLM вести до approve (автоапрув — лише за детермінованим порівнянням); рахувати VLM-виклики в ліміті інстансу; не зберігати diff у shouldAutoApprove. Зміна поведінки — через /spec.

VIS-SEC-062 — Таблиця Session росте без меж: рядки чистяться лише при новому вході того самого користувача, а PUT /users додає рядок без bcrypt і ліміту

Low · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L (4.3)` · **CWE:** CWE-770 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `backend/src/auth/auth.service.ts:46-48; backend/src/users/db/dbusers.service.ts:35-46; backend/src/users/users.controller.ts:231-238`

Опис. Єдине прибирання — `session.deleteMany({ userId, expiresAt < now })` у signToken того ж користувача; ні таймера, ні межі кількості входів. Завершені (endedAt) рядки лежать до expiresAt + наступного входу; рядки людей, що більше не входять, лишаються назавжди. Будь-який увійдений (зокрема guest) циклом PUT /users (один UPDATE + DELETE + INSERT, без bcrypt і без ліміту) додає скільки завгодно рядків за термін токена (типово доба).

Доказ.

```
auth.service.ts:46-48: `// rows of sign-ins whose tokens expired go as the person signs in again: no job needed\n await this.prismaService.session.deleteMany({ where: { userId: user.id, expiresAt: { lt: now } } });\n await this.prismaService.session.create(...)`
```

Вплив. Зростання бази й диска на хості, де поруч працює робочий VRT; деградація запитів. Потребує облікового запису, тому Low.

Рекомендація. Не створювати сесію в update() (див. попередню знахідку); періодично видаляти рядки з expiresAt < now або endedAt не null (за таймером), індекс на expiresAt (додавальний); межа активних сесій на людину (стару найдавнішу завершувати).

VIS-SEC-063 — Перевірка нестабільного тексту з кількома частинами може мовчки ніколи не спрацювати: кожна частина приймається, а разом вираз не компілюється

Low · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:L/A:N (3.5)` · **CWE:** CWE-754 · **OWASP:** A04:2021-Insecure Design · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `backend/src/ui-tree/variable-text.ts:29-32,40-60,63-65,78-83; backend/src/ui-tree/comparison-inputs.ts:169-175`

Опис. partsProblem() і rulePatternProblem() перевіряють кожен шаблон частини окремо (до 200 символів, до 20 частин). Під час перевірки partsRegex() склеює всі частини в один вираз. Якщо разом він перевищує ліміт RE2JS, compileLinear повертає 'unsupported', compile() → null, і checkVariableText мовчки повертає ok:false з причиною 'pattern' або 'text'. Правило збережене без помилки, problem:'pattern' у /structure не з'являється (rulePatternProblem перевіряє лише частини окремо), і кожен знімок з цим текстом іде на розгляд як зламаний. Це протилежне REQ-051-12, де причину треба показати.

Доказ.

```
/tmp/claude-0/-home-user-vision/82ab6902-a0af-5888-a773-e95df8c0504a/scratchpad/agent-C/vt2.ts, vt2-out.txt: 8 частин `(?:x*×90){1000}|\d+` (194 символи кожна) плюс фіксовані тексти: `partsProblem null | partsRegex null (does not compile)`; checkVariableText(parts,'Баланс 1 2 3 4 5 6 7 8 ₴') → {"ok":false,"reason":"pattern"}, хоча кожна частина тексту відповідає \d+.
```

Вплив. Хибні падіння перевірок і шум у розгляді. Через приховану причину editor не розуміє, чому правило не працює. Вплив на дані немає, сценарій штучний, тому Low.

Рекомендація. У partsProblem() компілювати й зведений вираз partsRegex(parts) і skeleton(parts) та відмовляти, якщо він не компілюється. У rulePatternProblem() перевіряти те саме, щоб збережене правило показало problem:'pattern'.

VIS-SEC-064 — API (будь-який його компроміс) пише rclone.conf, який хостовий rclone виконує без перевірки — обхід межі ADR-0013

Low · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:C/C:H/I:H/A:H (7.2)` · **CWE:** CWE-15 · **OWASP:** A04:2021-Insecure Design · **Компонент:** scripts · **Впевненість:** Medium · **Зусилля:** S

Де: `scripts/lib/vision-backup.sh:37-44,147-150; backend/src/backups/drive-auth.ts:103-119; docker-compose.demo.yml:115`

Опис. ADR-0013 обрав обмін файлами, щоб «API не може зачепити контейнери проду». Але API-контейнер має rw-монтування demo-data/backup-config і сам пише туди rclone.conf. Хостовий скрипт (cron) перевіряє лише, що у файлі є рядок `[vision-drive]` (`grep -qx`), і далі запускає `rclone` З ХОСТА (якщо rclone встановлено) з цим конфігом. rclone.conf — довільний конфіг rclone: можна задати інший type (наприклад sftp з опцією `ssh = <команда>` у rclone ≥1.64, local, alias) з довільними параметрами. Крім того, rcloneConf() вставляє clientId/clientSecret від адміна без екранування переносів рядка (trim лише по краях) — але для цього шляху Google має прийняти такі значення, тож практично досяжний лише варіант із уже скомпрометованим API.

Доказ.

```
vision-backup.sh:39 `grep -qx '\[vision-drive\]' "$VB_CONFIG_DIR/rclone.conf" 2>/dev/null || return 0` → :40 `RCLONE_CONFIG_DIR="$VB_CONFIG_DIR"`; :148-149 `if command -v rclone >/dev/null; then ... RCLONE_CONFIG="$RCLONE_CONFIG_DIR/rclone.conf" rclone "$@"`. docker-compose.demo.yml:115 `- ./demo-data/backup-config:/backup-config` (rw). drive-auth.ts:113-114 `client_id = ${connection.clientId}`, `client_secret = ${connection.clientSecret}`; backups.service.ts:170-171 лише `.trim()`.
```

Вплив. RCE в API (або зловмисний адмін за умови, що Google прийме значення) перетворюється на виконання команд на хості від імені користувача, який запускає vision-backup.sh, — того, хто має доступ до Docker і поряд робочий VRT (правило 1). Потрібен попередній компроміс API і хостовий rclone ≥1.64, тому Low.

Рекомендація. Хостовий скрипт не повинен брати конфіг rclone з теки, у яку пише API: API пише лише дані (drive.json з токенами), а скрипт сам генерує rclone.conf з фіксованим шаблоном (type=drive, scope=drive.file) і валідує поля (client_id за шаблоном `^[0-9A-Za-z._-]+$`, без \n). Або завжди запускати rclone у контейнері (як у fallback-гілці) без доступу до Docker. На API-стороні відхиляти clientId/clientSecret з керівними символами.

VIS-SEC-065 — frontend/package-lock.json: 162 пакети без resolved/integrity — npm ci не перевіряє їх цілісність

Low · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:N (5.9)` · **CWE:** CWE-494 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** frontend · **Впевненість:** High · **Зусилля:** S

Де: `frontend/package-lock.json (lockfileVersion 2)`

Опис. У lock-файлі інтерфейсу 162 з 846 записів не мають ні `resolved`, ні `integrity` (лише version/license). npm ci для них завантажує тарбол за версією з поточного реєстру (зокрема NPM_REGISTRY-дзеркала банку з frontend/Dockerfile:87) без звірки з хешем. Серед них 51 prod-залежність, що потрапляє в бандл інтерфейсу (classnames, clsx, prop-types, @socket.io/component-emitter, pako, readable-stream, stylis, react-material-ui-form-validator...). У backend/ і backend/prisma усі записи мають integrity.

Доказ.

```
python-аналіз lock: `162 846`, `no integrity 162`; приклад запису `{"version": "1.2.6", "dev": true,
"license": "MIT", "engines": {...}}` (node_modules/@aashutoshrathi/word-wrap); prod без integrity:
['node_modules/@socket.io/component-emitter','node_modules/classnames','node_modules/clsx',...
'node_modules/stylis'].
Перевірено окремо: frontend lockfileVersion 2 — 843 записи, 161 без integrity, з них 51 prod; backend і
backend/prisma — 0.
```

Вплив. Підміна пакета на дзеркалі/проксі з перепідписуванням TLS (VISION_EXTRA_CA_FILE) або компрометація реєстру для цих версій не буде виявлена — код потрапить у JS інтерфейсу (доступ до токена в localStorage та знімків).

Рекомендація. Перегенерувати lock (`rm -rf node_modules && npm install --package-lock-only` на npm ≥9, lockfileVersion 3) і додати в verify перевірку, що кожен запис з `node_modules/` має integrity.

VIS-SEC-066 — Postgres демо-стенду з паролем postgres/postgres досяжний з хоста через IP мережі Docker

Low · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N (7.7)` · **CWE:** CWE-1393 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** infra · **Впевненість:** Medium · **Зусилля:** S

Де: `docker-compose.demo.yml:49,142-144; .env.demo.example:11-12`

Опис. Порт бази не публікується (коментар :147), але дефолти POSTGRES_USER/PASSWORD=postgres у compose і в прикладі .env.demo, і API їх не перевіряє (secretProblems перевіряє лише JWT_SECRET і VISION_SECRET_KEY). На Linux контейнер у bridge-мережі досяжний з хоста за IP 172.x.y.z:5432 будь-яким локальним процесом/користувачем спільного з продом хоста без доступу до Docker.

Доказ.

```
docker-compose.demo.yml:143 `POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}`; .env.demo.example:12 `POSTGRES_PASSWORD=postgres`; backend/src/shared/server-secrets.ts:33 перевіряє лише [JWT_SECRET, VISION_SECRET_KEY].
```

Вплив. Будь-хто з shell на хості (зокрема сервіси сусіднього VRT) читає/змінює всю базу Vision: користувачі, хеші, знімки-метадані, токени звітів. Не перевірено на стенді (немає Docker), тому Medium confidence.

Рекомендація. Вимагати POSTGRES_PASSWORD (`${POSTGRES_PASSWORD:?}`), прибрати дефолт з прикладу (генерувати в vision-convert-env.py як JWT_SECRET), за потреби `internal: true` для мережі бази.

VIS-SEC-039 — Образи не містять LICENSE і NOTICE (Apache-2.0 §4(a), (d))

Info · **Стан:** не виправлено · **CVSS:** `n/a` · **CWE:** CWE-1104 · **OWASP:** n/a · **Компонент:** infra · **Впевненість:** High · **Зусилля:** S

Де: `['backend/Dockerfile', 'backend/prisma/Dockerfile', 'frontend/Dockerfile']`

Перевірка 2026-11-08: backend/Dockerfile:31-41, backend/prisma/Dockerfile (COPY . . з контексту backend/prisma — там немає LICENSE/NOTICE), frontend/Dockerfile:95-110 — жоден не копіює LICENSE/NOTICE; sdk-java/build.gradle — jar без META-INF/LICENSE, NOTICE — Корінь має LICENSE і NOTICE з атрибуцією VRT і deepmerge (NOTICE ок; vendor/deepmerge/license.txt присутній у всіх трьох копіях). backend/frontend/sdk-java мають LICENSE, але не NOTICE. Для внутрішнього використання — Info.

Рекомендація. COPY LICENSE NOTICE в кожен образ і в jar SDK; зберегти copyright-заголовки upstream і примітку про зміни в NOTICE.

VIS-SEC-040 — JWT без явного алгоритму, issuer і audience

Info · **Стан:** не виправлено · **CVSS:** `n/a` · **CWE:** CWE-345 · **OWASP:** A07:2021-Identification and Authentication Failures · **Компонент:** backend · **Впевненість:** Medium · **Зусилля:** S

Де: `['backend/src/auth']`

Перевірка 2026-11-08: backend/src/auth/auth.module.ts:13-19 і jwt.strategy.ts:14-18 без algorithms/issuer/audience; перевірка: токен HS512 тим самим секретом приймається `verifyAsync` (вивід: `HS512 accepted { email:'a', sid:'s', … }`) — Ризик низький (лише HS* з симетричним секретом, alg=none не проходить у jsonwebtoken 9.0.3).

Рекомендація. Задати `algorithms:['HS256']`, issuer, audience при verify.

VIS-SEC-041 — Editor може змінювати налаштування й видаляти будь-який доступний проєкт (зокрема ключ Gemini)

Info · **Стан:** не виправлено · **CVSS:** `n/a` · **CWE:** CWE-284 · **OWASP:** A01:2021-Broken Access Control · **Компонент:** backend · **Впевненість:** High · **Зусилля:** M

Де: `['backend/src/projects/projects.controller.ts:134,151']`

Перевірка 2026-11-08: backend/src/projects/projects.controller.ts:88-103 (PUT /projects, @Roles(admin, editor), @ProjectFrom(inBody('id'))), 105-114 (DELETE /projects/:id, admin, editor). У 051/052 не змінювалось; рішення власника в audit/PROGRESS.md немає. — Узгоджено з глобальною моделлю ролей 042. Потрібне рішення власника або специфікація про роль на рівні проєкту.

Рекомендація. Розглянути права на рівні проєкту (owner) поверх глобальних ролей. Потребує специфікації.

VIS-SEC-056 — StripUnknownFieldsPipe не чистить вкладені DTO: зайві ключі областей ігнорування зберігаються в JSON

Info · **Стан:** не виправлено · **CVSS:** `n/a (Info)` · **CWE:** CWE-915 · **OWASP:** A08:2021-Software and Data Integrity Failures · **Компонент:** backend · **Впевненість:** High · **Зусилля:** S

Де: `['backend/src/shared/strip-unknown-fields.pipe.ts:33', 'backend/src/test-runs/dto/update-ignore-area.dto.ts:11-14', 'backend/src/test-runs/dto/create-test-request.dto.ts:38-43', 'backend/src/test-runs/test-runs.service.ts:459', 'backend/src/test-runs/test-runs.service.ts:725']`

Перевірка 2026-11-08: backend/src/shared/strip-unknown-fields.pipe.ts:33 — фільтр і далі лише верхнього рівня (у 051 додано тільки ListLengthPipe). backend/src/test-runs/test-runs.service.ts:731 `ignoreAreas: JSON.stringify(ignoreAreas)` і :459 `tempIgnoreAreas: JSON.stringify(createTestRequestDto.ignoreAreas)` зберігають елементи як прийшли. — Тепер ≤20 областей на запит (051), тож обсяг сміття обмежений. Виправляючи, збережіть поле `id`: інтерфейс на нього покладається (replaceAreaWithElement, test-runs.service.ts:615; WhatsCompared.tsx `item.area.id`). Тобто мапити на {id?, x, y, width, height}, а не на 4 поля.

Рекомендація. Перед збереженням мапити області на {x,y,width,height} (як normalizeMarks) або зробити пайп рекурсивним для властивостей з @ValidateNested/@Type. Задокументувати в backend/AGENTS.md, що пайп діє лише на верхньому рівні.

VIS-SEC-057 — Застарілі облікові дані й чужа IP-адреса в документації та тестах

Info · **Стан:** не виправлено · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:N/A:N (0.0)` · **CWE:** CWE-1188 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** infra · **Впевненість:** High · **Зусилля:** S

Де: `['DEPLOY.md:640-642', 'sdk-java/README.md:89', 'sdk-java/src/test/resources/vision_config_example.json:2', 'sdk-java/src/test/java/ua/vision/sdk/VisionConfigTest.java:52']`

Перевірка 2026-11-08: DEPLOY.md:654 «если не заданы — `visual-regression-tracker@example.com` / `123456`» (compose ставить admin@vision.local, docker-compose.demo.yml: 131-132); sdk-java/README.md:89 `"apiUrl": "[http://162.243.161.172:4200](http://localhost:4200)"`; sdk-java/src/test/java/ua/vision/sdk/VisionConfigTest.java:52,62 — Рядок у DEPLOY.md змістився з 642 на 654.

Рекомендація. Виправити DEPLOY.md (admin@vision.local, вимога змінити пароль); замінити IP на localhost/example у README і тестах SDK.

VIS-SEC-067 — Успадкований TLS-конфіг ssl/ без заголовків безпеки й CSP

Info · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N (3.1)` · **CWE:** CWE-1021 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** infra · **Впевненість:** High · **Зусилля:** S · **Пов'язано з:** VIS-SEC-011

Де: `ssl/default.conf:1-21, ssl/nginx.conf`

Опис. Єдиний TLS-конфіг у дереві (upstream VRT, слухає 5000 ssl) не має server_tokens off, X-Frame-Options, nosniff, Referrer-Policy, COOP, HSTS і не підключає /tmp/vision-csp.conf. Він ніде не використовується (Dockerfile копіює лише nginx/), але як «готовий TLS» може бути взятий при розгортанні — і тоді CSP/HSTS з 052 зникнуть саме там, де з'явиться https. Блок 1 робить 301 з `$host` (Host з запиту).

Доказ.

```
ssl/default.conf:11-20 `listen 5000 ssl; ssl_certificate ...; location / { root ...; try_files ... }` — жодного add_header; grep по DEPLOY.md/README/compose — посилань на ssl/ немає.
```

Вплив. Ризик неправильного розгортання TLS без заголовків безпеки.

Рекомендація. Видалити ssl/ або переписати як include поточного default.conf + `listen 8443 ssl` + HSTS безумовно на https; описати TLS у DEPLOY.md.

VIS-SEC-068 — Звіт аудиту з відкритими вразливостями й репро закомічено і потрапить в експорт у git банку

Info · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N (3.1)` · **CWE:** CWE-538 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** scripts · **Впевненість:** High · **Зусилля:** S

Де: `audit/ (git ls-files); scripts/export-without-history.sh:36`

Опис. audit/REVIEW-PROMPT.md каже «Артефакти в audit/ (не комітити)», але audit/findings.json, report.pdf/md, raw/repro-.txt, raw/agent-*.json відстежуються git. export-without-history.sh робить `git archive` усіх відстежуваних файлів, тож детальний перелік невиправлених вразливостей з кроками відтворення потрапить у репозиторій банку для всіх, хто має до нього доступ.

Доказ.

```
`git ls-files audit` → audit/PROGRESS.md, audit/findings.json, audit/raw/agent-A-2026-11-07.json, …, audit/raw/repro-001-patch-builds.txt; .gitignore не містить audit/.
```

Вплив. Розкриття карти вразливостей ширшому колу, ніж власник і безпека банку.

Рекомендація. Додати audit/ у .gitignore і `git rm --cached -r audit` (окрім, можливо, REVIEW-PROMPT.md), або виключити audit/ у export-without-history.sh (`export-ignore` у .gitattributes); звіт передавати окремим захищеним каналом.

VIS-SEC-069 — Образ інтерфейсу віддає source maps

Info · **Стан:** нова · **CVSS:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (5.3)` · **CWE:** CWE-540 · **OWASP:** A05:2021-Security Misconfiguration · **Компонент:** frontend · **Впевненість:** High · **Зусилля:** S

Де: `frontend/vite.config.ts:42 (`sourcemap: true`); frontend/Dockerfile:99`

Опис. Збірка робиться з sourcemap: true, і build/ цілком копіюється в nginx, тож *.js.map з повним вихідним кодом інтерфейсу (з коментарями) доступні без входу. Код не секретний (власний репозиторій), але банківські сканери позначають це як розкриття інформації.

Доказ.

```
vite.config.ts `build: { outDir: "build", sourcemap: true, ... }`; Dockerfile `COPY --from=builder /app/
build .`; nginx default.conf location / без обмежень на .map.
```

Вплив. Мінімальний: полегшує розвідку.

Рекомендація. `sourcemap: 'hidden'` або видаляти *.map в образі; або `location ~ \.map$ { return 404; }`.

7. Дані за межами контуру та LLM

Канал	Що виходить	Стан на `fe7a82c`
Gemini — VLM-рушій при прийомі	знімки екранів	прийнятий ризик власника (VIS-SEC-003), не оцінювався
Gemini — AI на вимогу, чернетки багів	тексти дерева, назви перевірок, знімки	`AI_PROVIDER=none` не вимикає особисті ключі (VIS-SEC-016); guest може запускати (VIS-SEC-026)
Jira	опис бага, вкладення-знімки	лише `https:443` з `VISION_JIRA_HOSTS`, без переадресацій, токен особистий — закрито (008, 048); wiki-розмітка без екранування (054)
Google Drive	бекап бази й знімків без шифрування	рішення власника (ADR-0012), не оцінювався; новий ризик на межі хоста — VIS-SEC-064
Ollama	тексти й знімки	адреса лише з конфігу адміністратора (`OLLAMA_BASE_URL`), SSRF немає
Публічні звіти	білий список полів	без змін, ADR-0022; токени відкритим текстом (025)

Правило 6 (LLM лише на вимогу) — у коді на вимогу дотримано: AI-сервіс викликається лише з маршрутів `/ai/*`; при прийомі — лише VLM-рушій, якщо проєкт сам його обрав (виключено як 003). Prompt injection у вердикт VLM (032) і автоапрув за ним (055) лишаються Low.

8. Комерційне використання, ліцензії й банківський контур

Ліцензії. Модель — внутрішнє використання автором і його компанією, без дистрибуції (рішення власника). Порахунок по встановлених пакетах (`raw/licenses-2026-11-08.json`): у поставці — MIT, ISC, Apache-2.0, BSD, BlueOak, 0BSD, PSF-2.0 (`argparse`), OFL-1.1 (шрифти); `pause` і `color-convert` без поля ліцензії — MIT за README і LICENSE. EPL-2.0 `elkjs` (Prisma Studio) вирізається з обох образів (`backend/Dockerfile:29`, `backend/prisma/Dockerfile:20`) — FP. MPL-2.0 (`axe-core`, `lightningcss`) — лише dev. Нова залежність `re2js` — MIT.

Apache-2.0. `LICENSE` і `NOTICE` у корені з атрибуцією VRT і deepmerge — гаразд; в образах і jar їх немає (VIS-SEC-039, Info). Бренд «Visual Regression Tracker» поза `LICENSE/NOTICE` лишився лише там, де мова про робочий VRT, з якого Vision копіює дані, і в одному застарілому рядку `DEPLOY.md:654` (057).

Чужий контент: фото реальних людей у `backend/test/image*.png` (042); чужа IP у README SDK (057).

Контрольні питання для банку (без юридичних висновків — потребує юриста/DPO):

- **Аудит-лог безпеки:** окремого журналу немає; входи, зміни ролей, видача доступу, паролі від адміна — лише частково в загальному лозі Nest. Для банку це зазвичай обов'язкова вимога.

- **Строки зберігання:** демо-compose вимикає очищення (`TASKS_CLEANUP_ENABLED=false`), тож знімки з ПІБ і номерами карток зберігаються безстроково. Потрібна політика строків.
- **Видалення даних суб'єкта ПД** (Закон України «Про захист персональних даних», GDPR): процедури немає.
- **PCI DSS:** якщо на знімках бувають повні PAN, Vision потрапляє в скоуп PCI. Тестові дані застосунків мають бути синтетичними — це питання процесу QA, а не коду.
- **Шифрування:** at rest — лише секрети (044); знімки й база — на рівні диска хоста. In transit — TLS-термінація банку (власний https відкладено).
- **Моніторинг і реагування:** немає метрик безпеки й алертів (невдалі входи, сплески 5xx, OOM).

9. Roadmap

До перенесення (блокери й дефекти щойно зроблених виправлень):

1. VIS-SEC-058 — межа розміру програми RE2 (оцінка добутку повторів у `translate`, відмова з причиною), DTO `@IsString @MaxLength(300)` для `selector-matches`, кеш без незбережених виразів і за байтами; тест REQ-051-11 з `(x*){1000}`.
2. VIS-SEC-043 — `pickableElements` без повного XPath на кожен варіант; `/structure` у воркер з тайм-аутом; тест на 10 000 рядків з однаковим `resource-id`.
3. VIS-SEC-059 — `PUT /users` переписує токен із поточним `sid` або завершує поточний; e2e «update → logout → обидва токени 401».
4. VIS-SEC-060 — поточний пароль для `PUT /users/password` (і зміни email у режимі БД); нова вимога — специфікація.
5. VIS-SEC-061 — спільна межа областей ігнорування після злиття.
6. Конфігурацією поставки: VIS-SEC-010 (обов'язковий `DEFAULT_USER_PASSWORD` ≥ 12 без друку в лог), VIS-SEC-016 (`AI_PROVIDER=none` вимикає й особисті ключі) або явне рішення власника по них.
7. VIS-SEC-068 — прибрати `audit/` з експорту в контур (`.gitattributes export-ignore` або окремий канал).

30 днів: 004/046 (решта прийому — у воркер), 017 (пароль від адміна лише локальним), 062 (чистка сесій), 065 (lock інтерфейсу з integrity), 066 і 019 (паролі баз), 064 (rclone.conf генерує скрипт), 053 (межа кроків XPath), 063, 026, аудит-лог безпеки.

90 днів: решта Low/Info; 021 (guard.py як запобіжник, а не контроль); 030/031 (pinning CI і Gradle); політика строків зберігання і процедура видалення ПД.

10. Додатки

10.1. Сканери і розбір хибних спрацювань

Інструмент	Результат	Розбір
`npm audit` (prod і dev × backend, backend/prisma, frontend)	0 / 0 / 0	—

Інструмент	Результат	Розбір
semgrep, 977 правил	1210 спрацювань, з них 1020 `i18next-key-format`	i18n і стиль — FP. `path-join-resolve-traversal` у `backups.service.ts` — імена файлів-константи (`FILES.*`), у `hdd.service.ts` — шлях перевіряється `resolve` у межах кореня. `detect-non-literal-regexp` у `linear-regex.ts:345` — лише перевірка синтаксису, виконання на RE2; у `variableText.helper.ts` — у браузері користувача. `gcm-no-tag-length` у `secret-box.ts` — тег з власного шифротексту в базі (див. 024). `no-replaceall` у LDAP — справжня, VIS-SEC-050. `missing-image-version` — образи за digest через ARG — FP. `github-actions-mutable-action-tag` — справжня, VIS-SEC-030.
bandit (CLI)	2 Medium B310, 2 Low	URL з конфігурації користувача CLI; `subprocess` зі списком аргументів — FP; редирект з `apiKey` — VIS-SEC-028
shellcheck	43 (2 error SC2148 — бібліотеки без shebang), як і минулого разу	стиль, FP
regex-сканер секретів	78 збігів	плейсхолдери в `.env.example`, тестових compose і `control-scenario.sh` — FP; ключів і сертифікатів у дереві немає (`scripts/lib/no-extra-ca.pem` порожній)
ліцензії	див. розділ 8	`elkjs` — FP (вирізається при збірці)

10.2. Тести

Набір	Результат
Unit бекенду (Node 24.11)	929 / 929
E2E бекенду на стенді	91 / 91 (9 наборів, зокрема `sessions`, `security-fixes`, `startup-secrets`)
Unit фронтенду	274 / 274
`scripts/check-docs.py`	Documentation is consistent

10.3. Перевірені області без зауважень

- Ізоляція проєктів 042: 88 маршрутів і socket.io — кожен з `@ProjectFrom`/`@NoProjectData`, тест маршрутів покриває всі контролери; IDOR, API-ключ A → B, масиви id, multipart — без обходу.
- Правило 7: рядок `User` ніде не віддається цілком; `Session` не віддається жодним маршрутом.

- Ін'єкції: `$queryRaw` лише параметризований, `Unsafe` немає; JQL — `jiraKey` за regex; `dangerouslySetInnerHTML` немає; open redirect і header injection немає.
- Прийом PNG (сигнатура, кілька IHDR, interlaced, обрізаний чанк, межа канви, ліміти multer).
- Jira після 051: лише `https:443`, `user:pass@`, `redirect: 'manual'`.
- `/images` (basename, `./..`), nginx 404 на `/static/imageUploads/`.
- CSP в nginx і API, HSTS лише по https (підробка `X-Forwarded-Proto` безпечна), `env.sh`.
- Сесії 052: вихід, зміна пароля, пароль від адміна, видалення, зміна ролі — в API і сокетах.
- SSO: `nonce` вкладки, `state`/PKCE у httpOnly-cookie, переадресація лише на `APP_FRONTEND_URL`.
- Секрети при старті (051): `JWT_SECRET` і `VISION_SECRET_KEY` ≥ 32, не з прикладу, різні.

10.4. Артефакти

`audit/findings.json` (усі записи зі статусами), `audit/report.md|html|pdf`, `audit/threat-model.md`, `audit/sbom.cdx.json`, `audit/raw/*-2026-11-08*` (агенти A–D, сканери, тести, відтворення `repro-008…010`), `audit/PROGRESS.md`. Попередні ітерації — `report-2026-11-07.*`, `findings-2026-11-07.json`, `report-2026-11-05.*`.